Binary Search Trees

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

The End is Near!

- Second midterm exam, Thu June 5th 11am, this room
  - Same format as first midterm, closed notes, no electronics
  - Not cumulative (but builds on earlier material)
  - Material until Thursday inclusively is covered
  - Will release a practice exam

- Final project
  - Initial deadline tonight
    - Highly recommend getting (at least!) `locate_all` working
    - Keep a backup of your last non-crashing version!
    - Submit that to get actual feedback
  - 2nd try (+ 2nd design doc) due Tue June 3rd (next week)
  - 3rd try (+ 3rd design doc) due Tue June 10th (finals week)

Dictionaries, Revisited

- Possibly **the** most broadly useful ADT.
  - We used 3 in our data design example alone!

- Hash tables are a fine default, but not always the best!
  - $\mathcal{O}(1)$ operations, but only *expected*
    - Need enough buckets
    - Need good distribution of data across buckets
    - Still $\mathcal{O}(n)$ worst case
  - And $\mathcal{O}(n)$ for rehashing anyway…
  - May waste space on empty buckets (constant factor)

Dictionaries, Revisited

- Possibly **the** most broadly useful ADT.
  - We used 3 in our data design example alone!

- Hash tables are a fine default, but not always the best!
  - $\mathcal{O}(1)$ operations, but only *expected*
    - Need enough buckets
    - Need good distribution of data across buckets
    - Still $\mathcal{O}(n)$ worst case
  - And $\mathcal{O}(n)$ for rehashing anyway…
  - May waste space on empty buckets (constant factor)

- There are other options! Today: *binary search trees*
  - Binary search trees = binary trees + invariant
  - With some smarts, can get $\mathcal{O}(\log n)$ operations *worst case*!
  - And no big jumps in space usage! (vs rehashing)

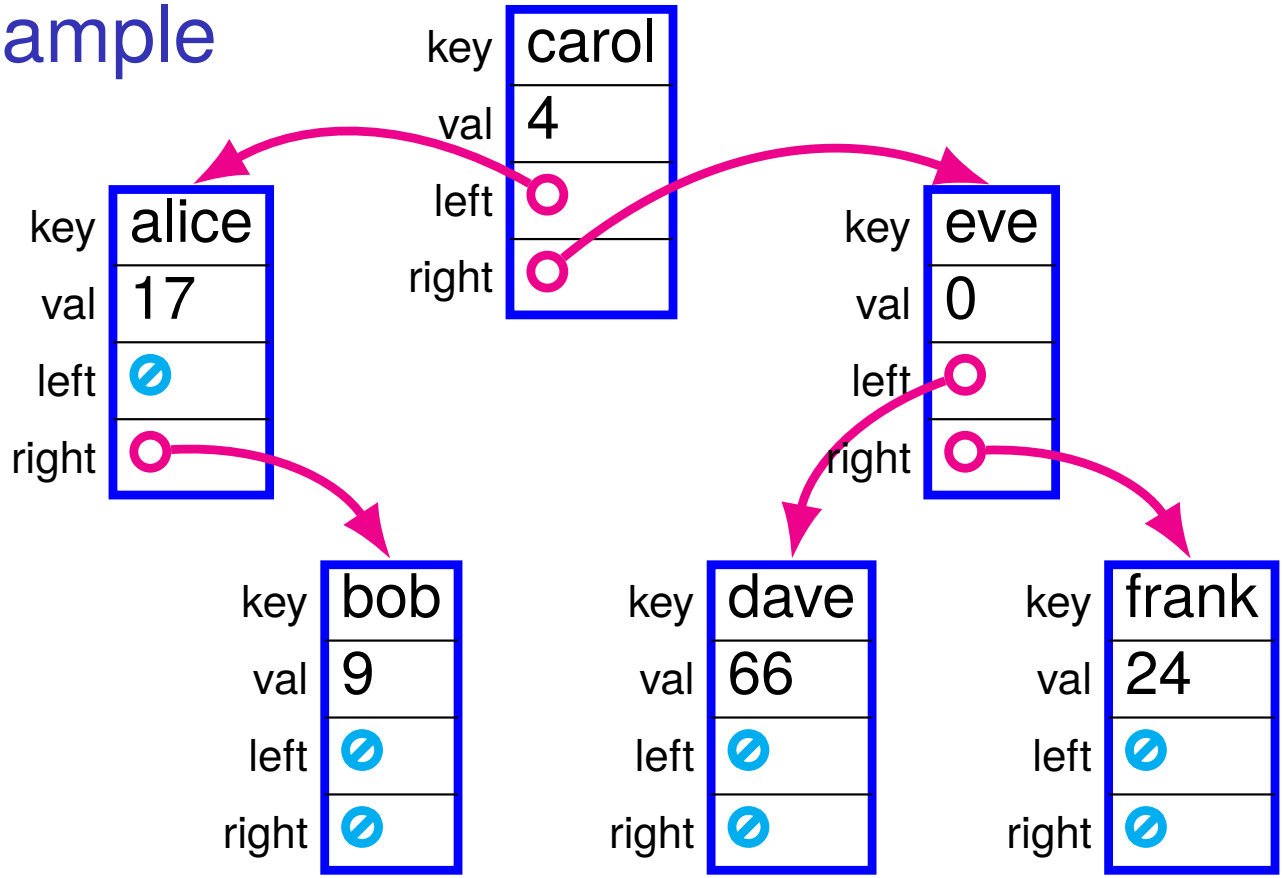
Enter the Binary Search Tree (BST)

A *binary search tree* stores elements *in order* in a *linked* data structure (in this case, a binary tree).

- *In order* means we can do binary search
  - Like with a sorted array!
- *Linked* means we can easily insert new elements
  - Like with a linked list!

Best of both worlds!

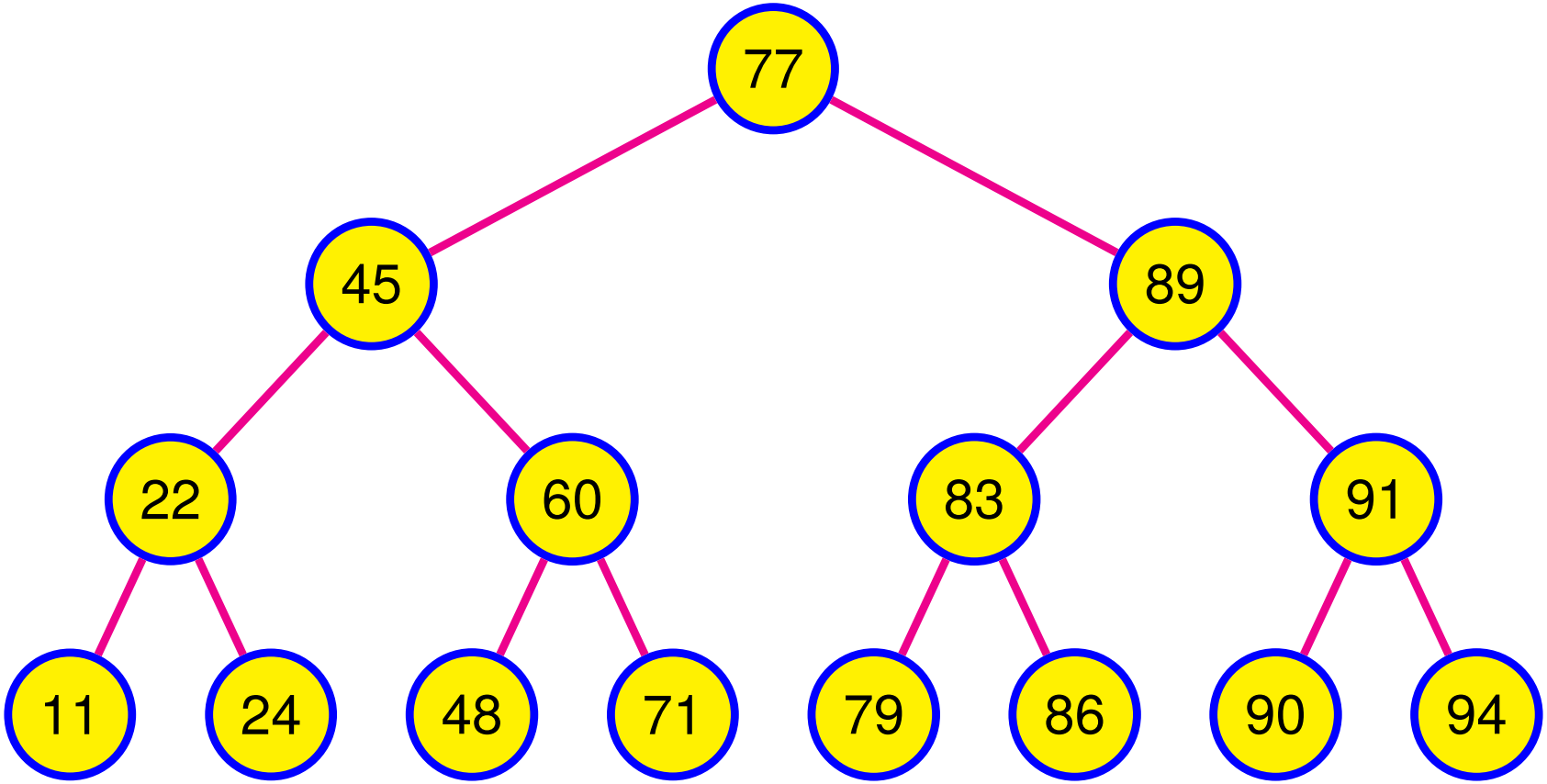
BST example



- **In order**: from *left* to *right*, keys are in increasing order
  - **Invariant**: For each node *n*,
    - every node in our *left* subtree has a key *less than* *n*'s key,
    - m.m. *right* subtree and *greater than*
- **Linked**: store the tree as linked node structs
  - New key-value pair $\rightarrow$ new node.

BST lookup

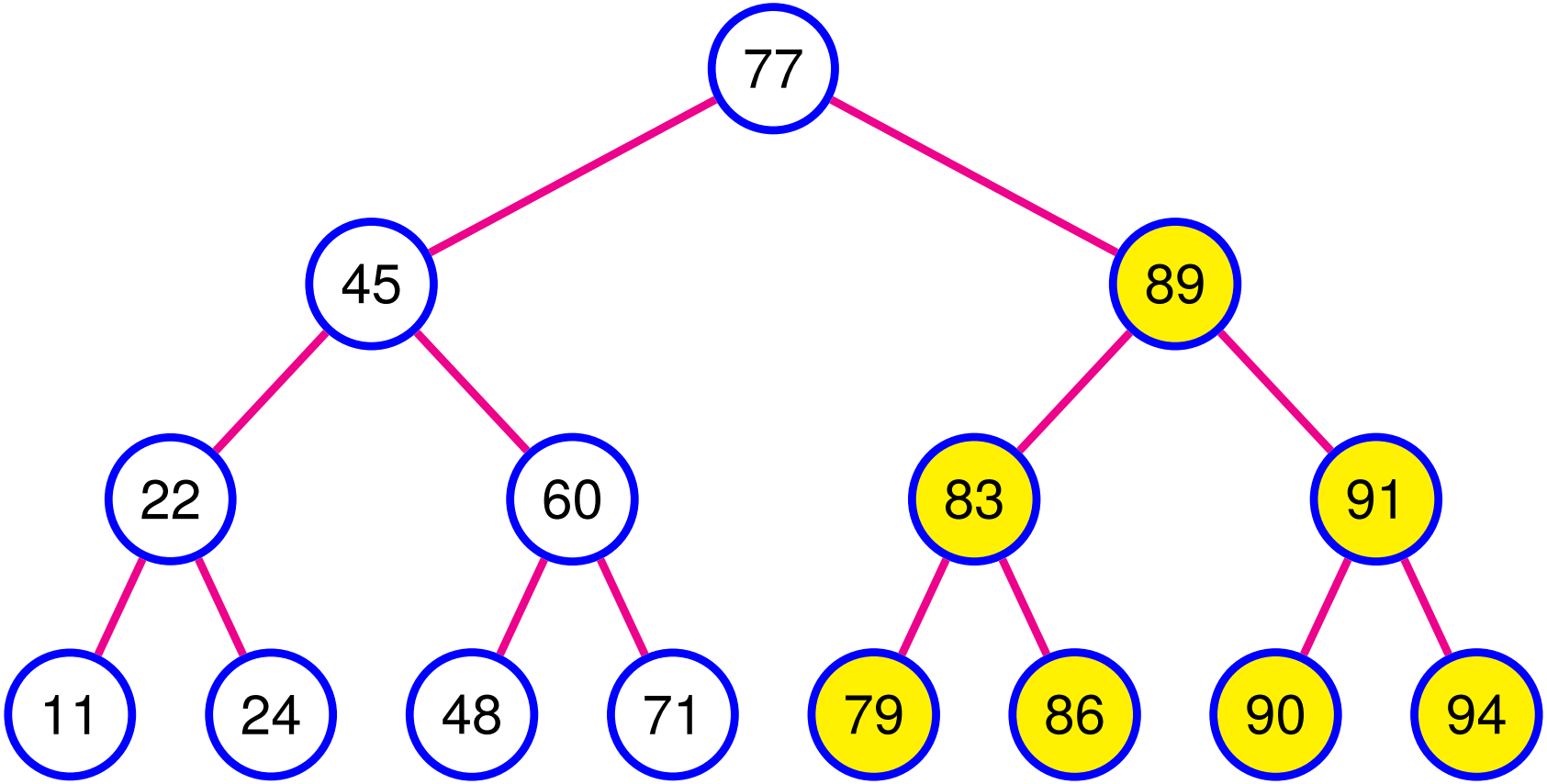
Say we're looking up 86.



BST lookup

Say we're looking up 86.

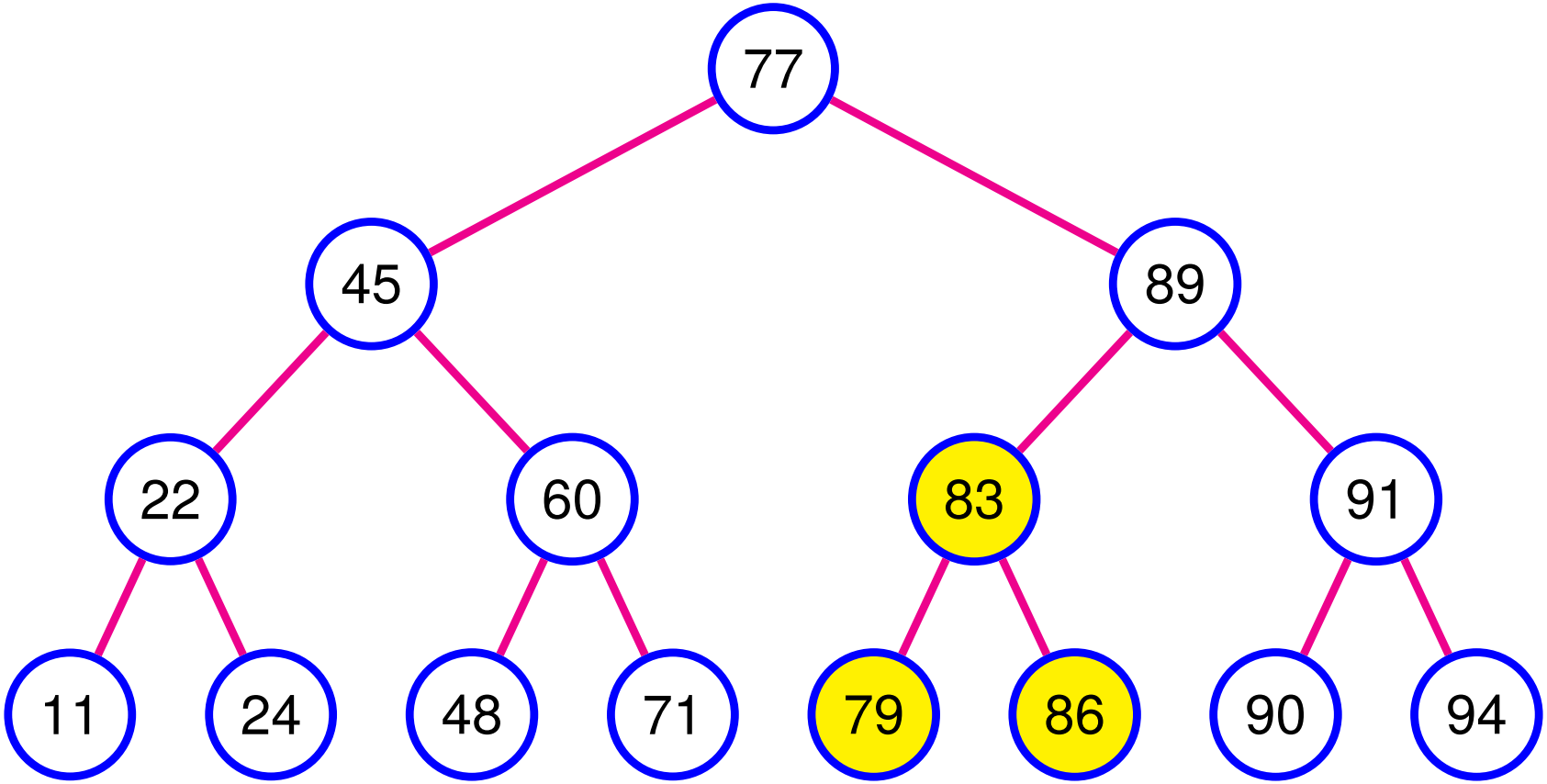
We cut one subtree each step.



BST lookup

Say we're looking up 86.

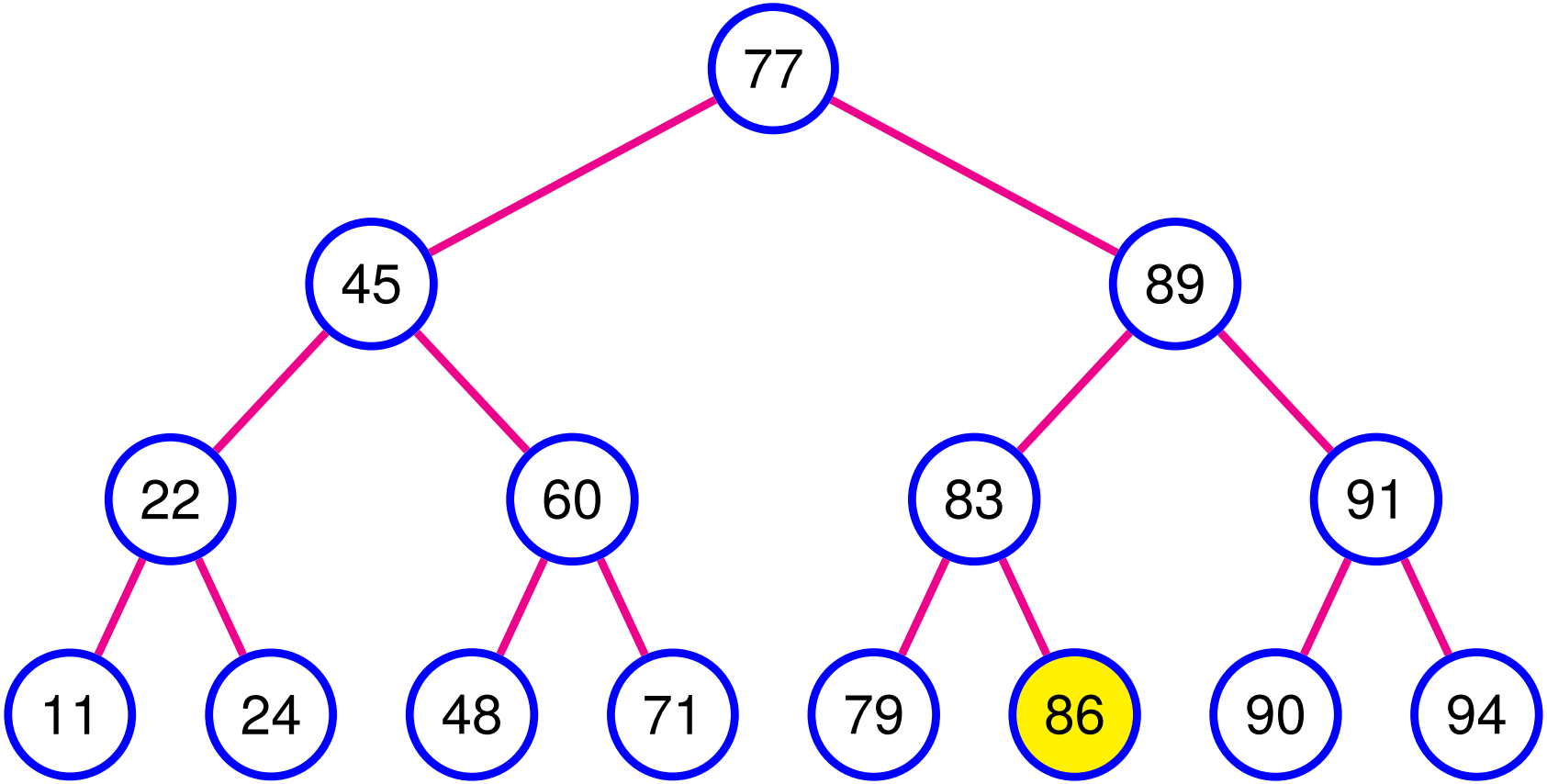
We cut one subtree each step.



BST lookup

Say we're looking up 86.

We cut one subtree each step.



BST lookup algorithm

```
Function get(root, key) is
    Input: A node root and a key key
    Output: A value, or nothing

    curr ← root;
    while curr is not None do
        if key < curr.key then
            curr ← curr.left
        else if key > curr.key then
            curr ← curr.right
        else
            return curr.val
        end
    end

    return None
end
```

Binary search, we meet again?

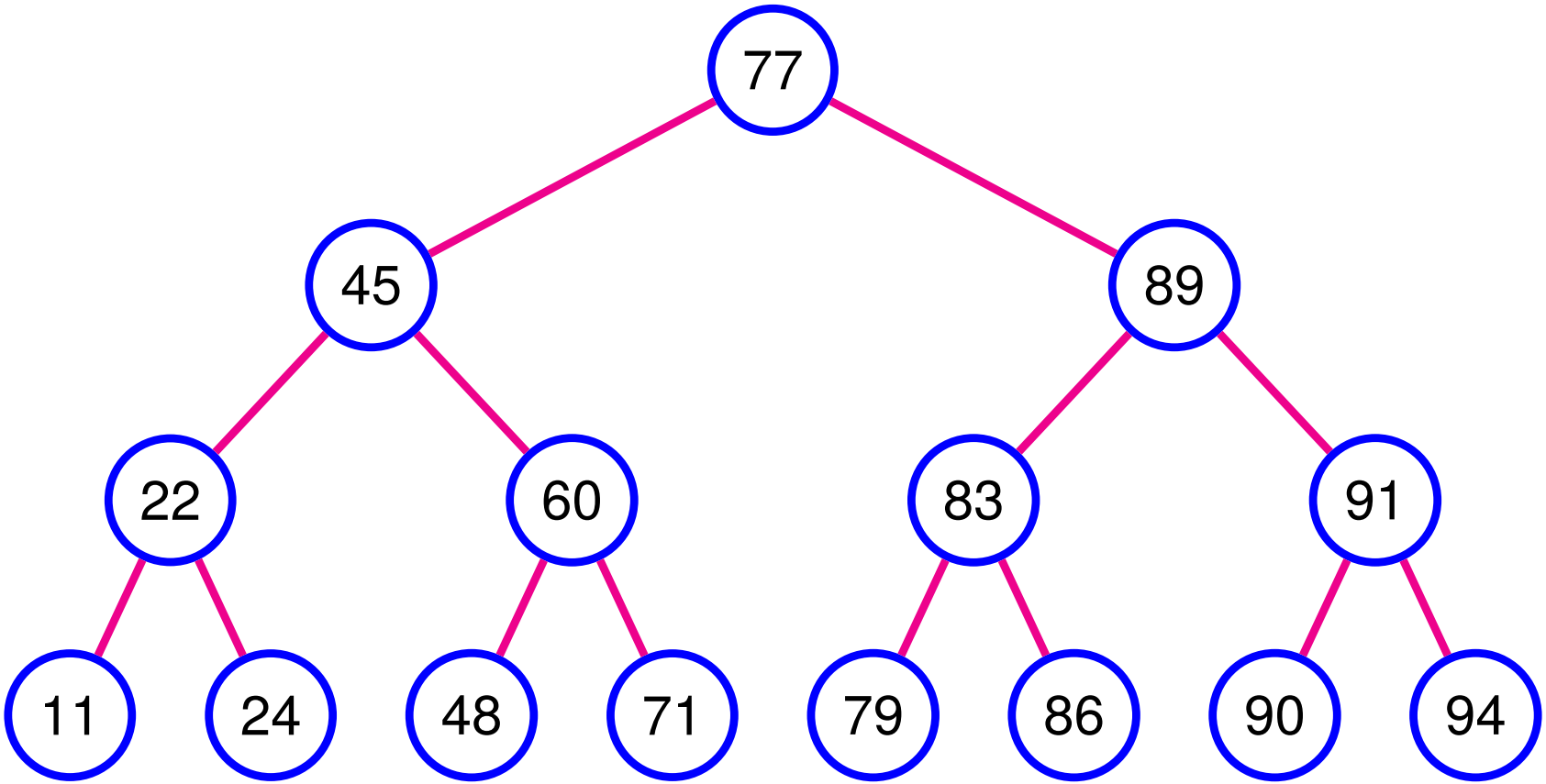
With subtrees instead of intervals.

We cut one subtree each step.

BST delete

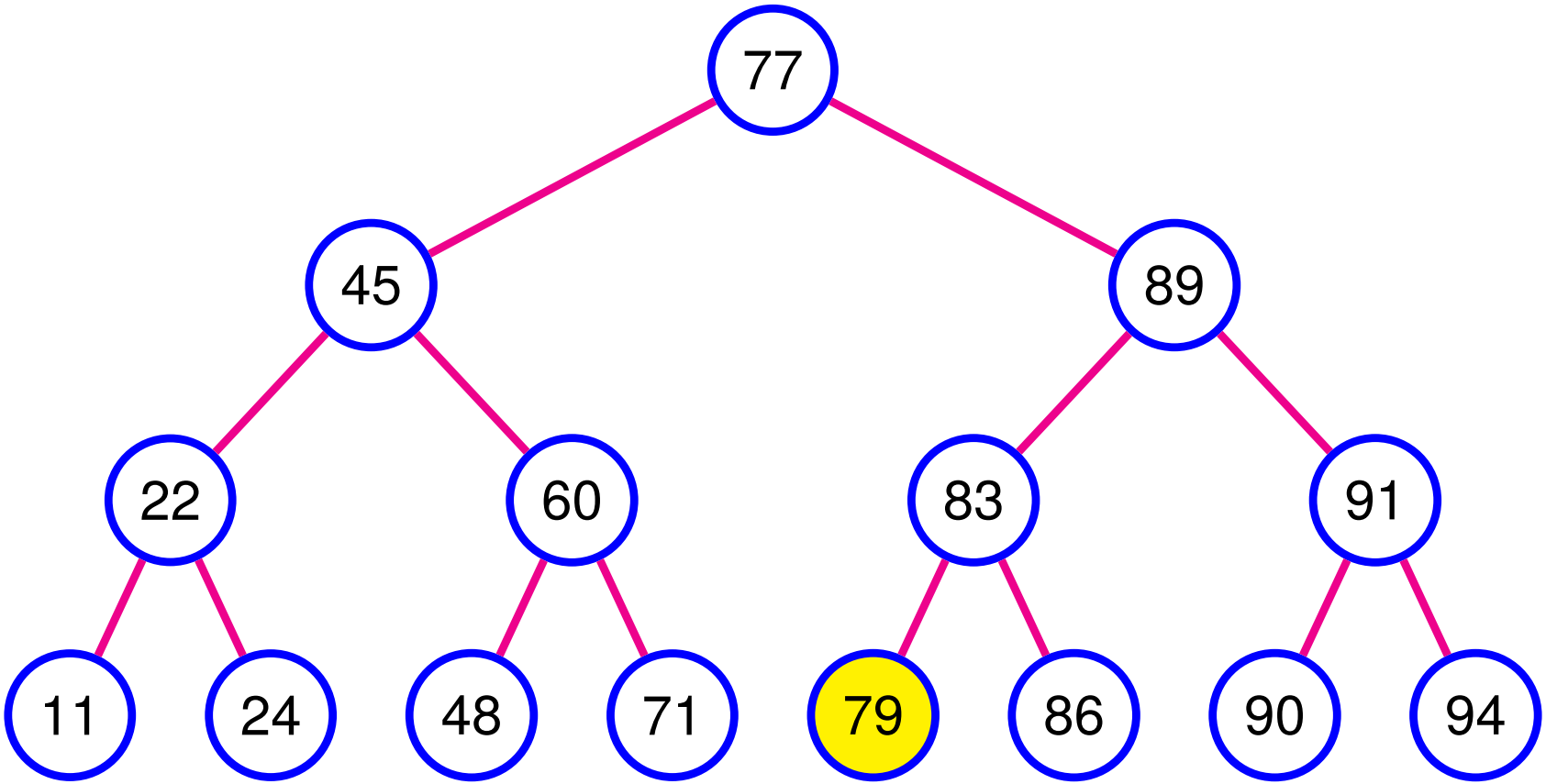
Less interesting than insert.

So let's get it out of the way.



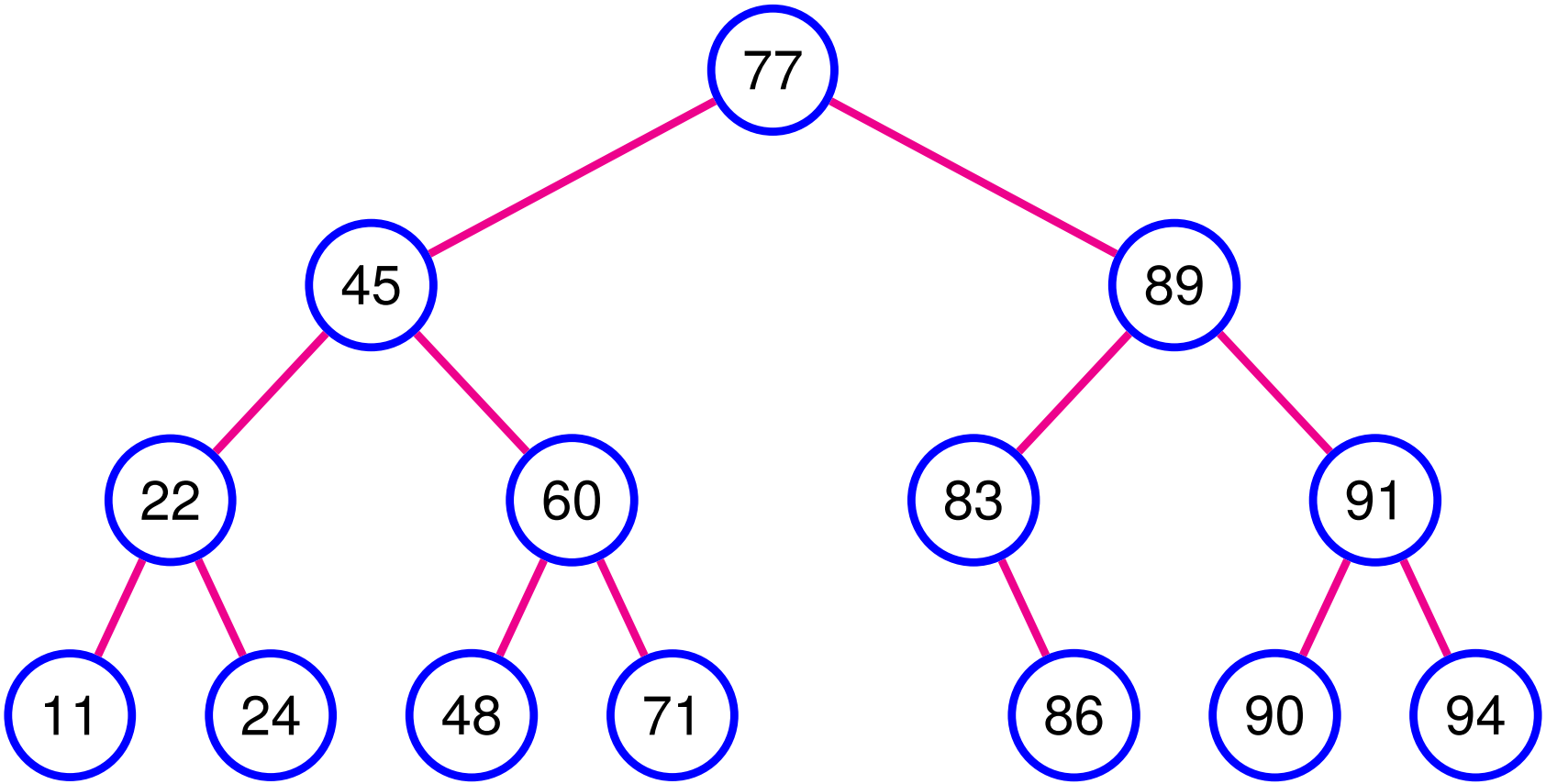
BST delete

For leaves: replace with None.



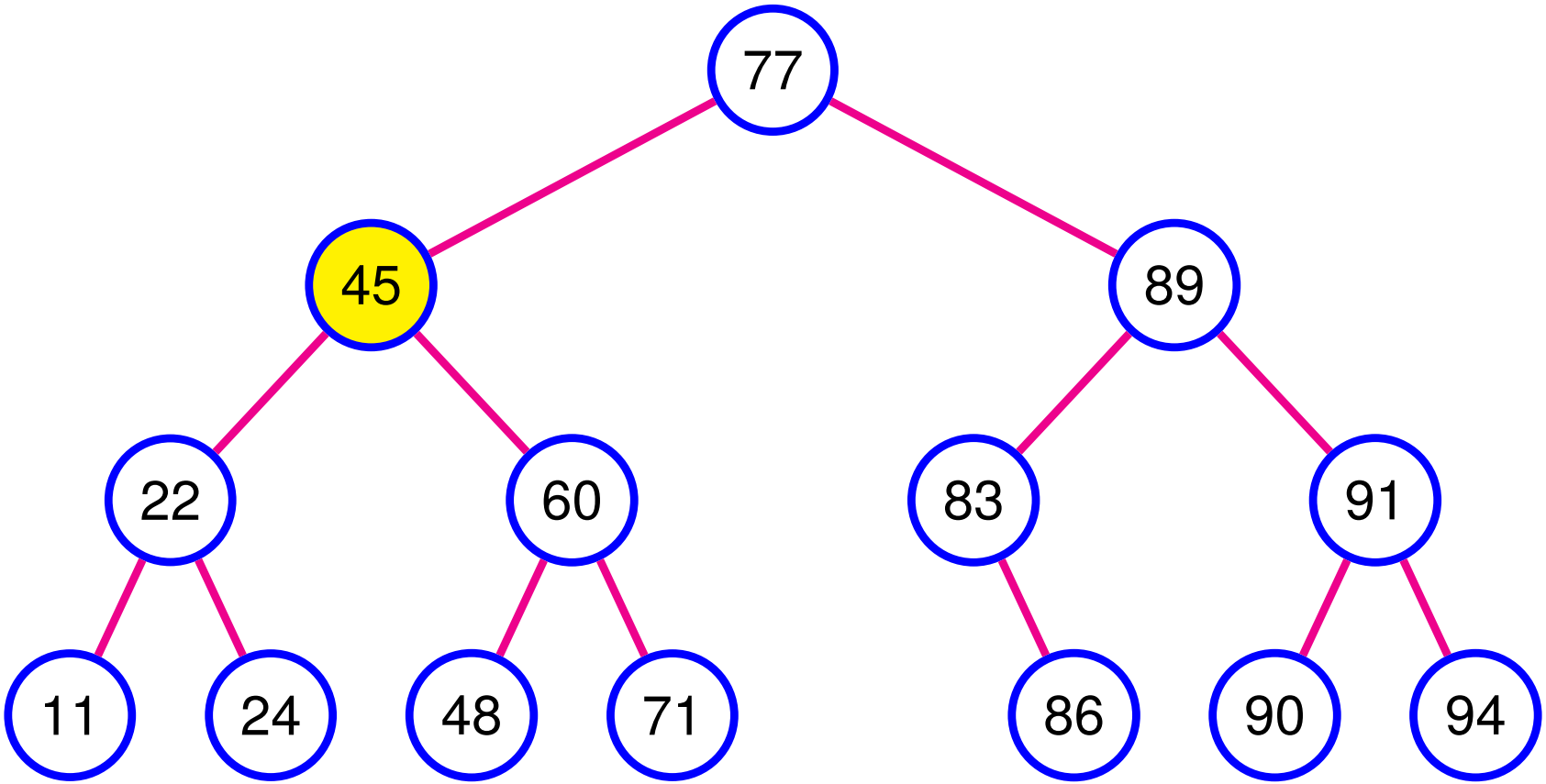
BST delete

For leaves: replace with None.



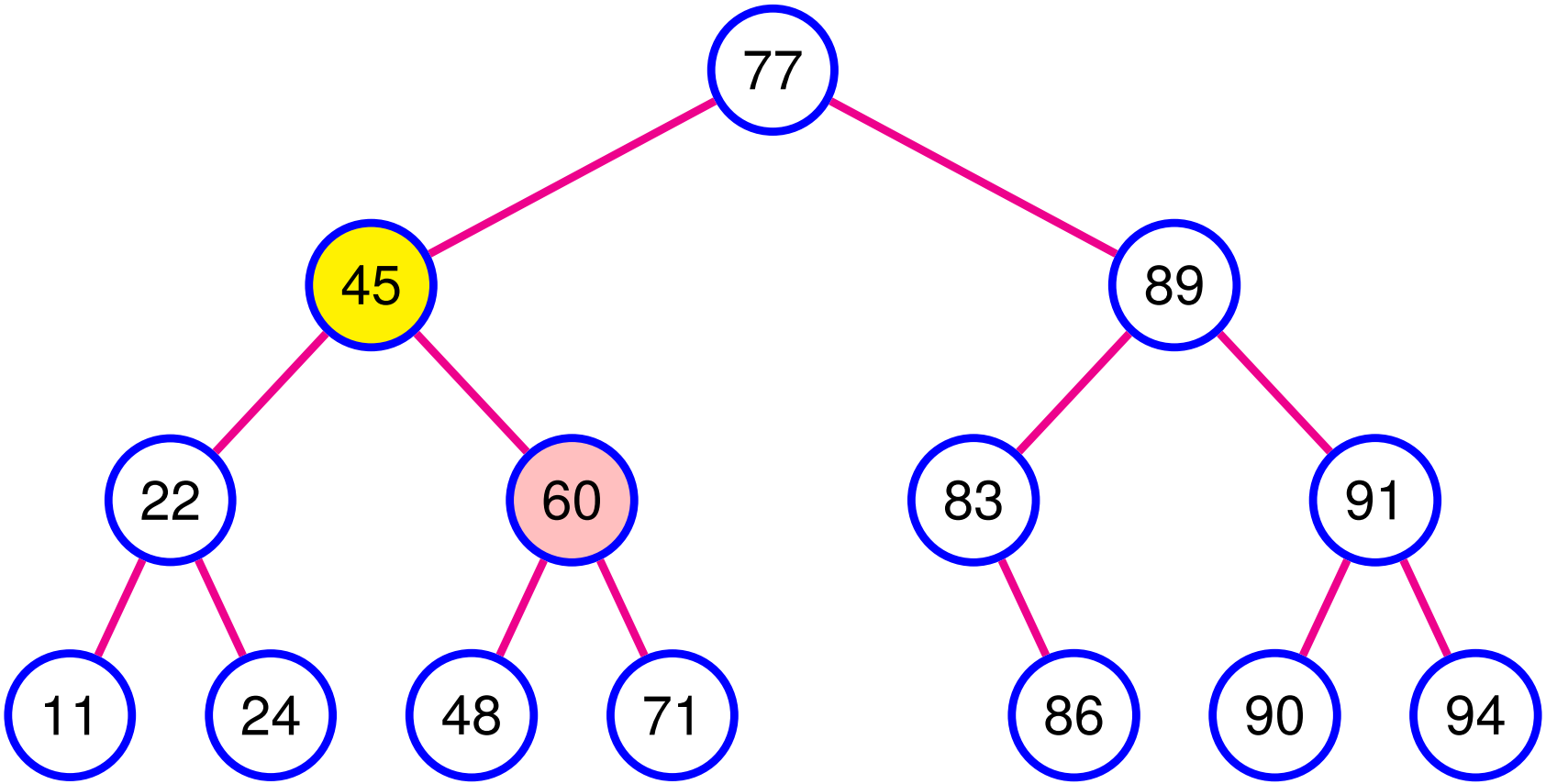
BST delete

For internal nodes: find “next” node.
I.e., leftmost desc. of right child.



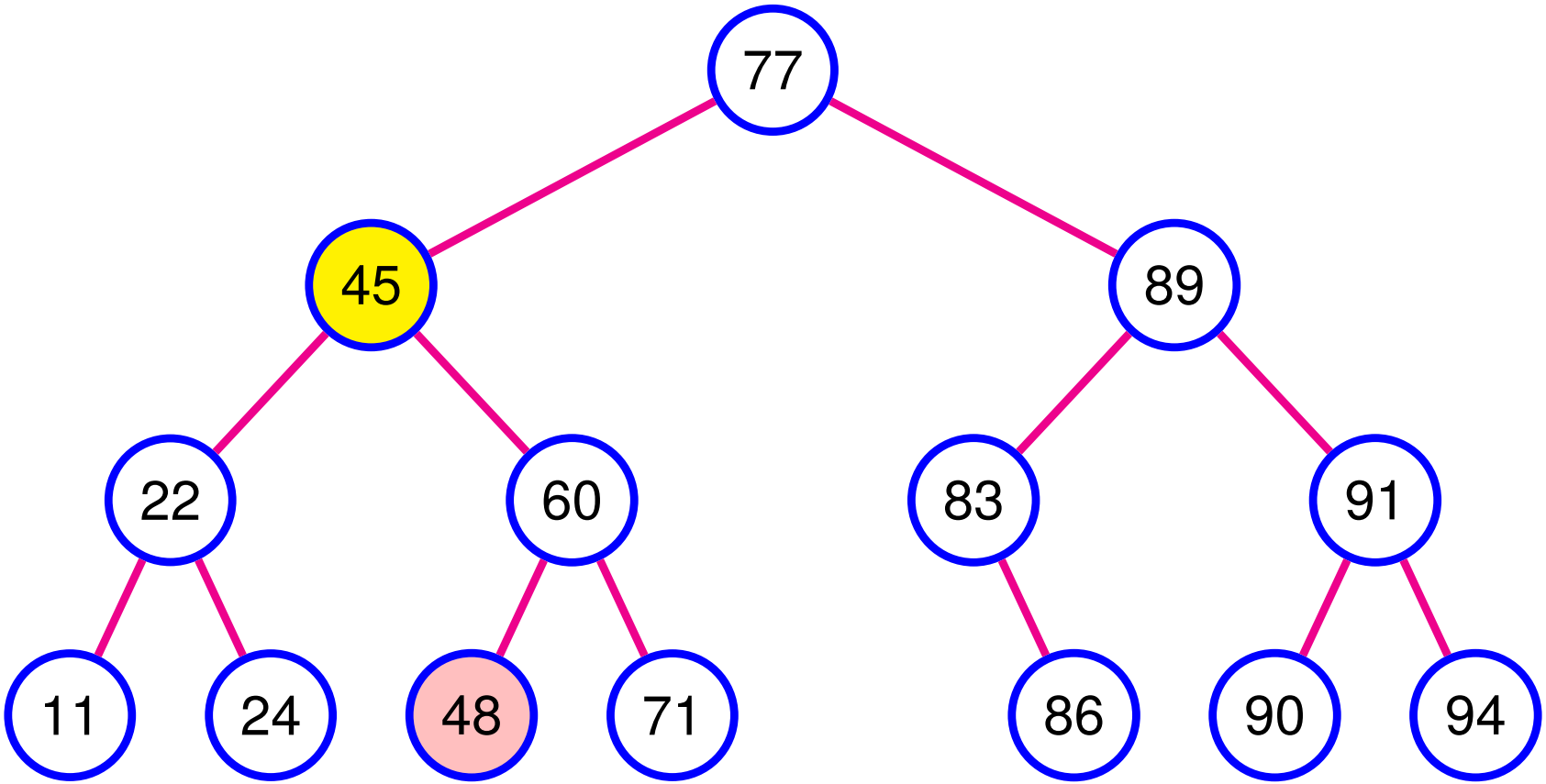
BST delete

For internal nodes: find “next” node.
I.e., leftmost desc. of right child.



BST delete

For internal nodes: find “next” node.
I.e., leftmost desc. of right child.

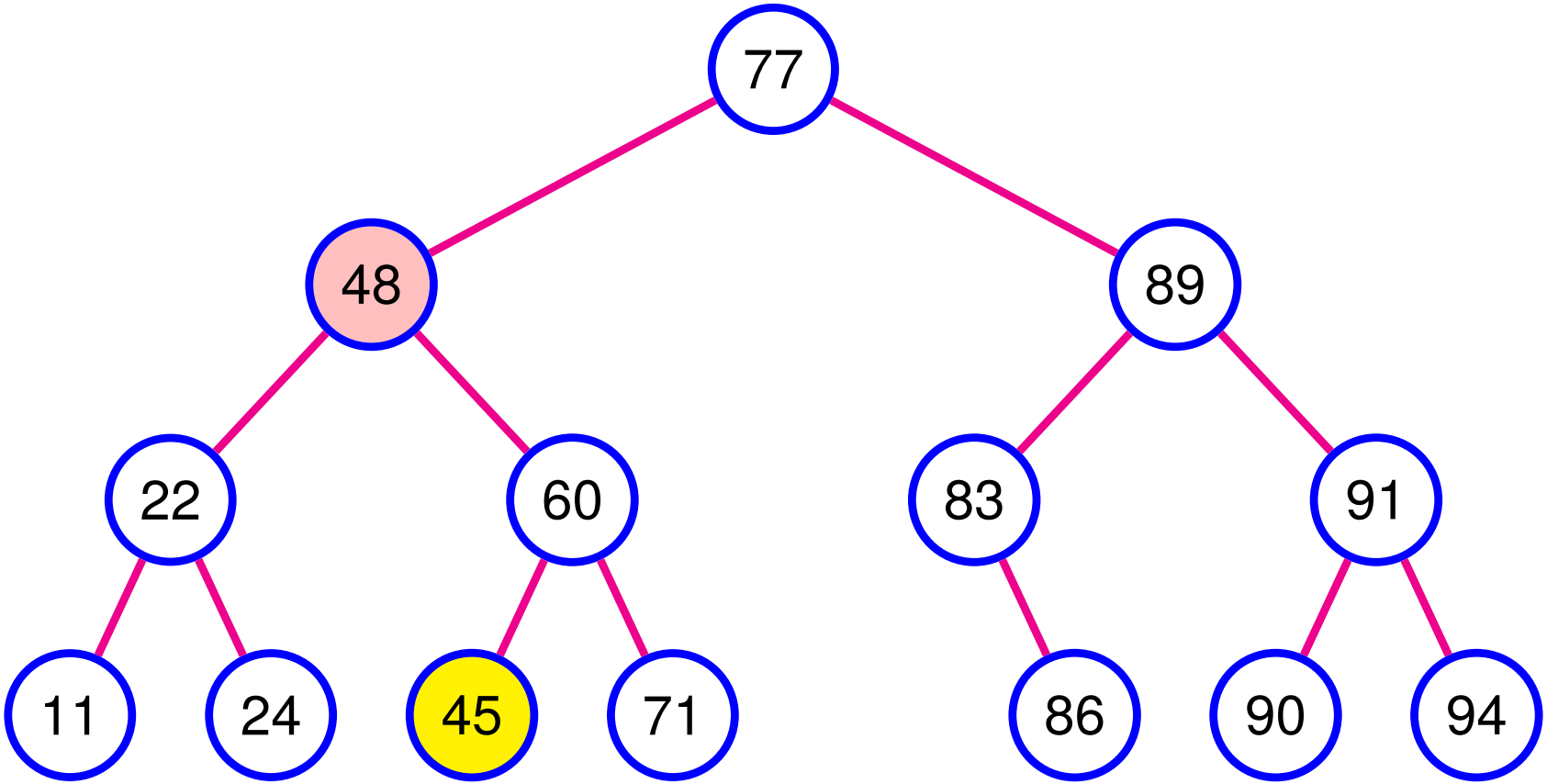


BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.

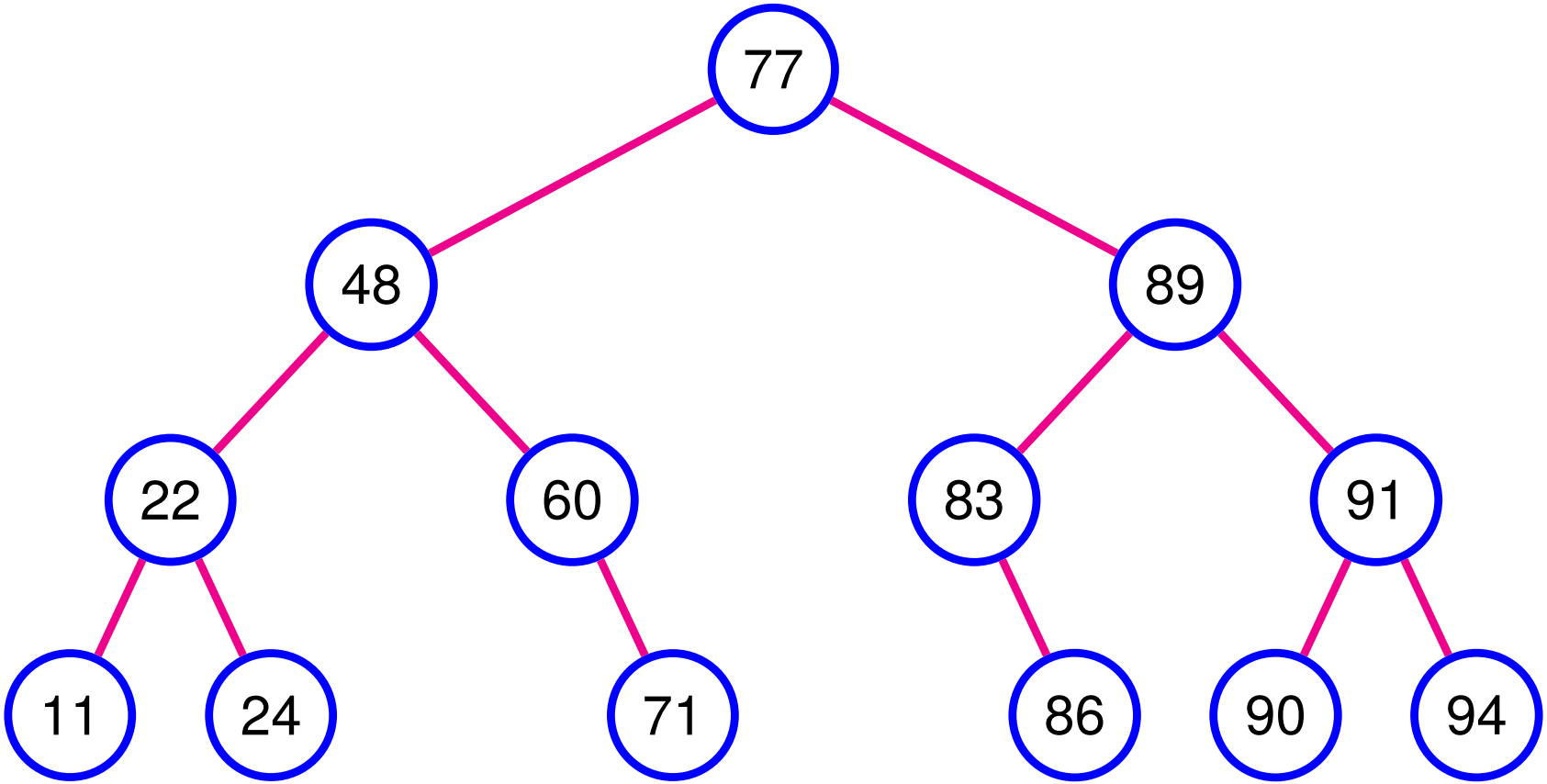


BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.

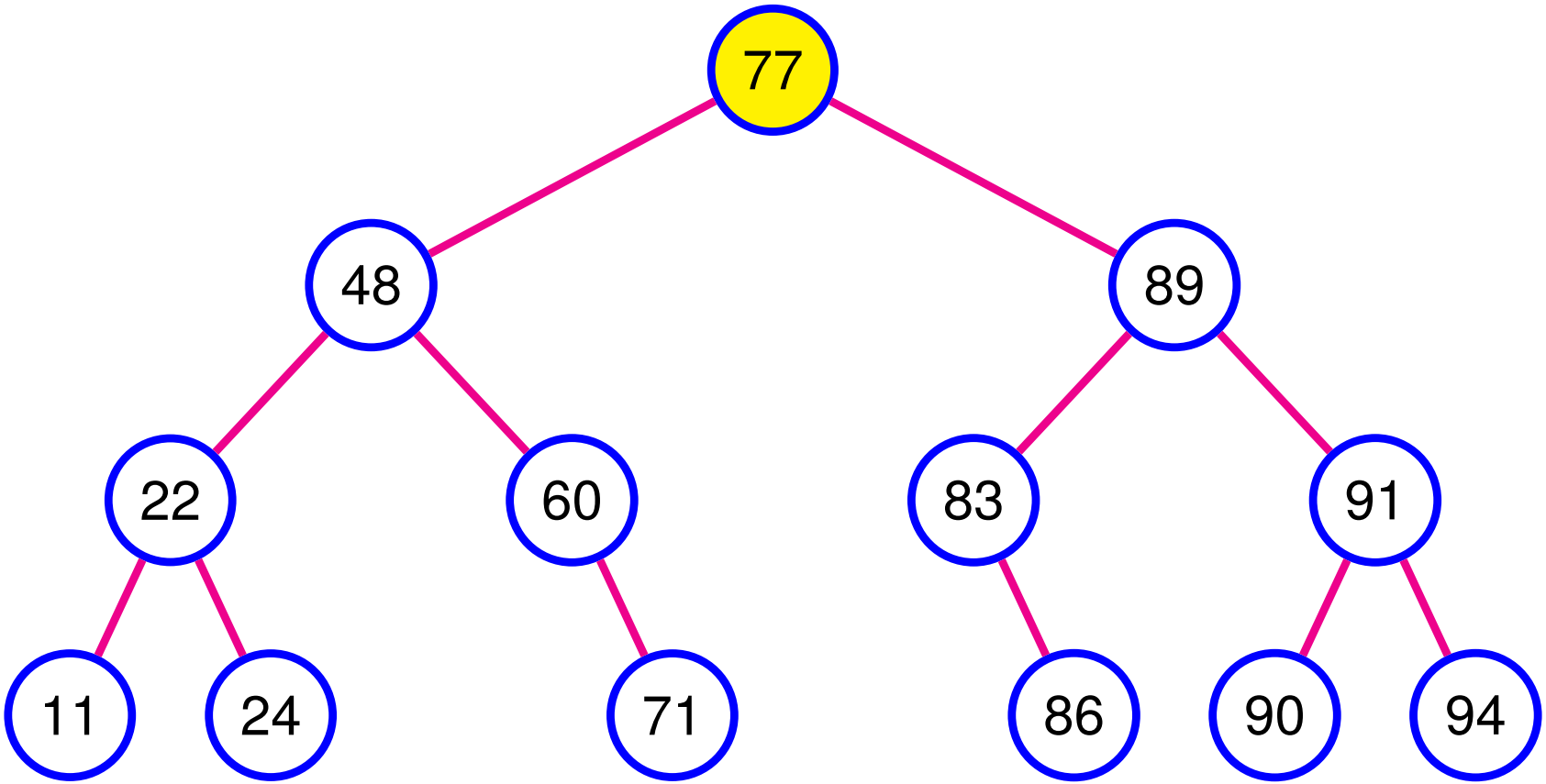


BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.

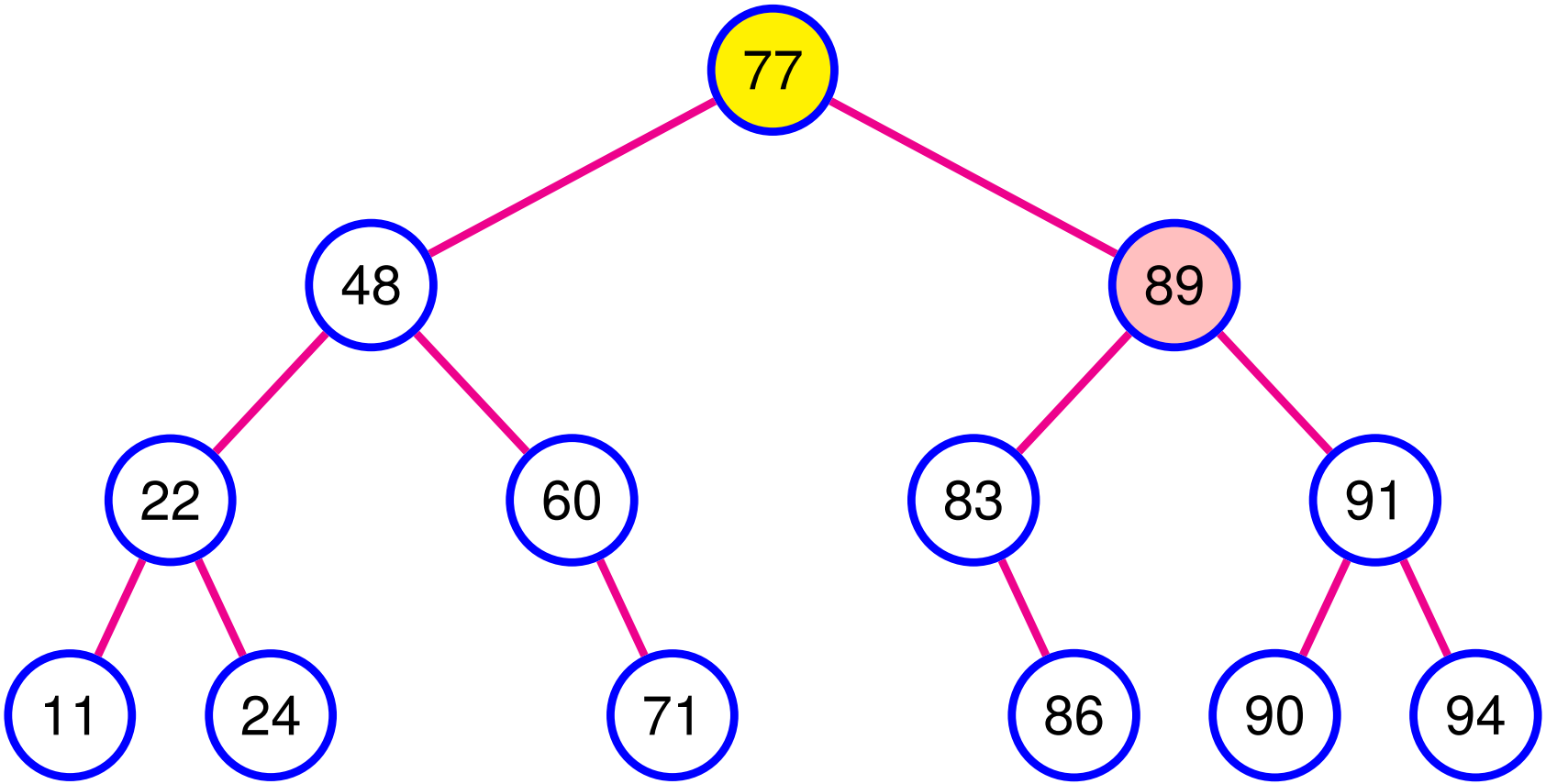


BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.

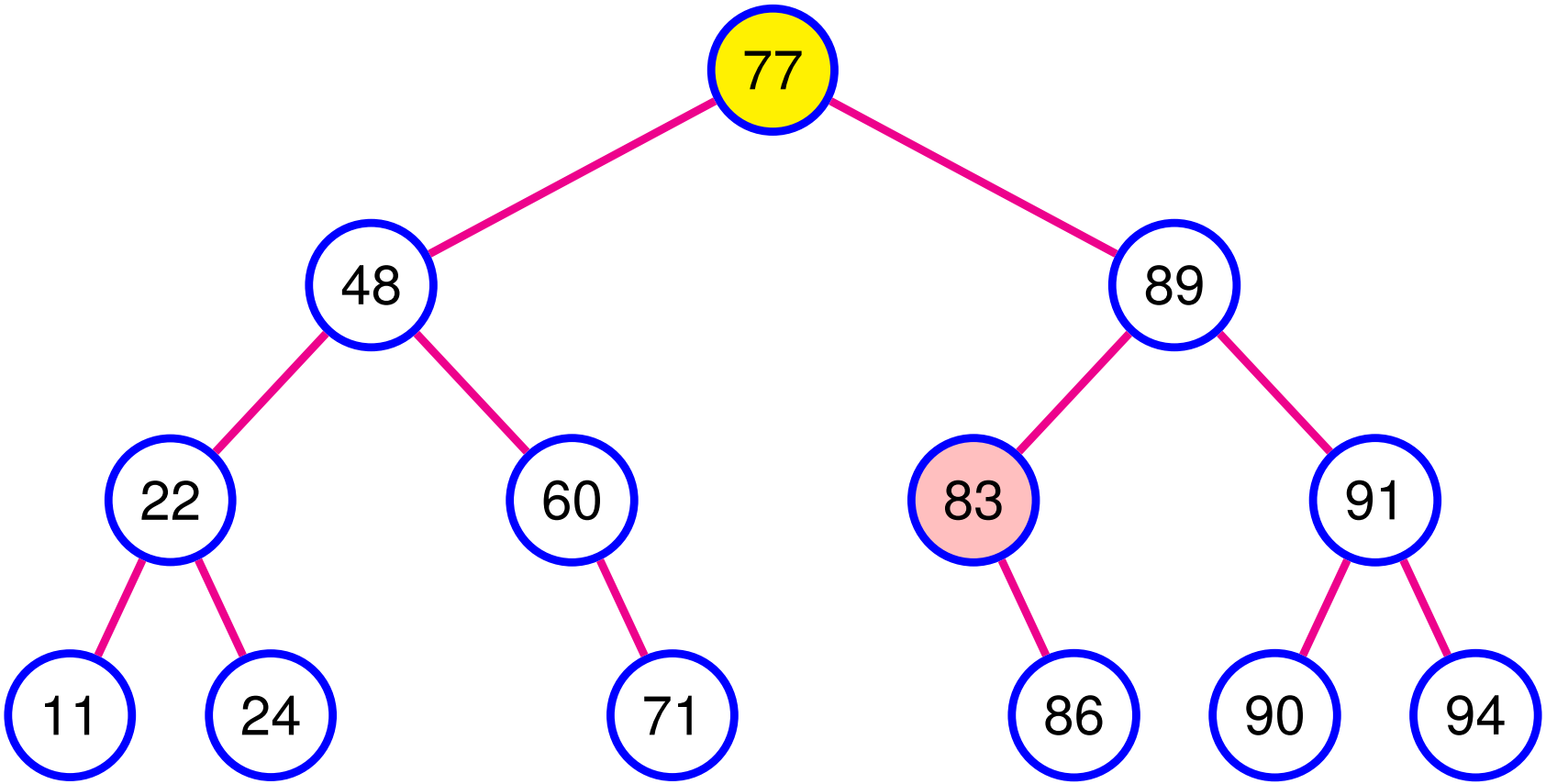


BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.



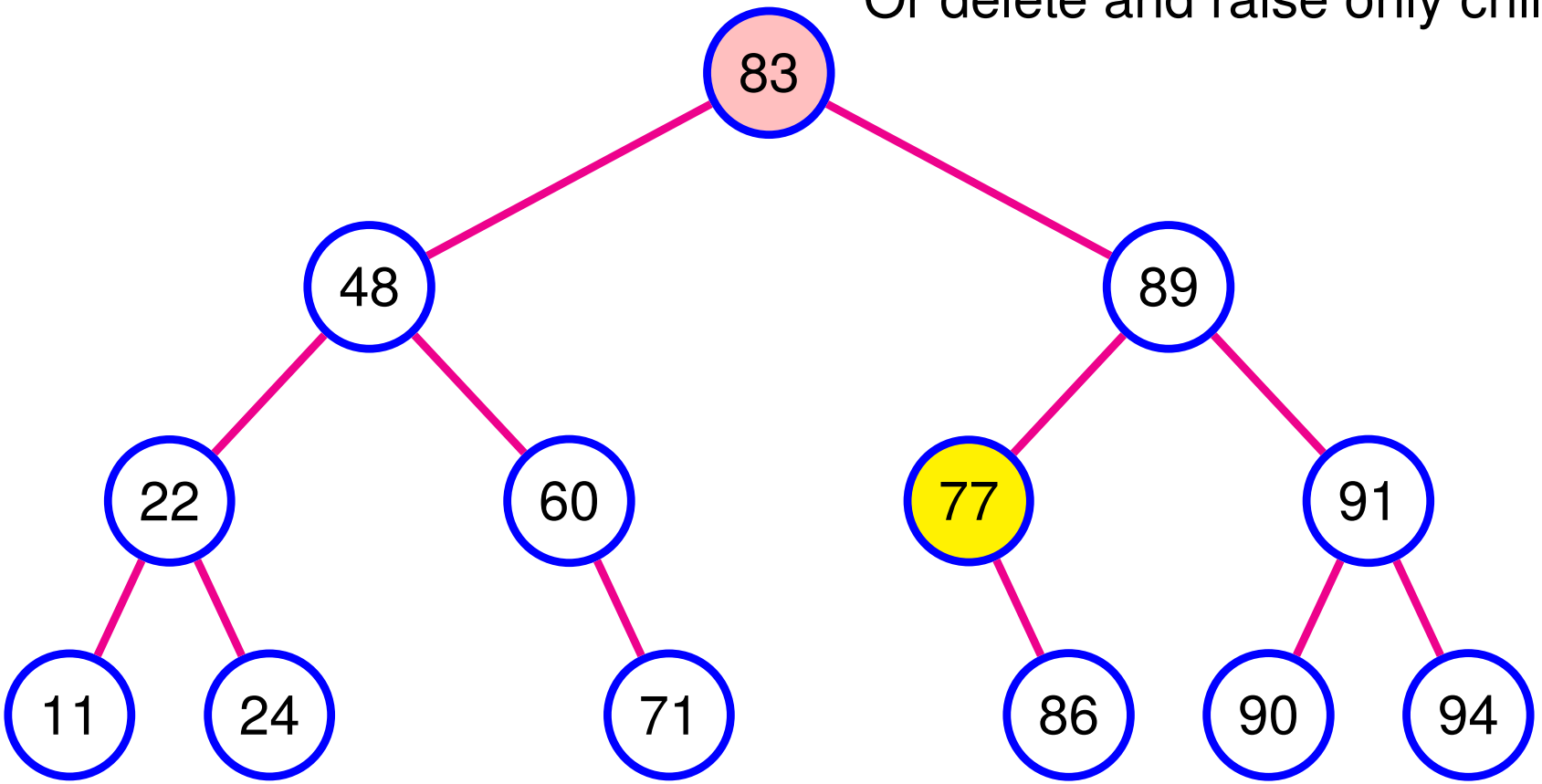
BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

Swap them, then delete leaf.

Or delete and raise only child.



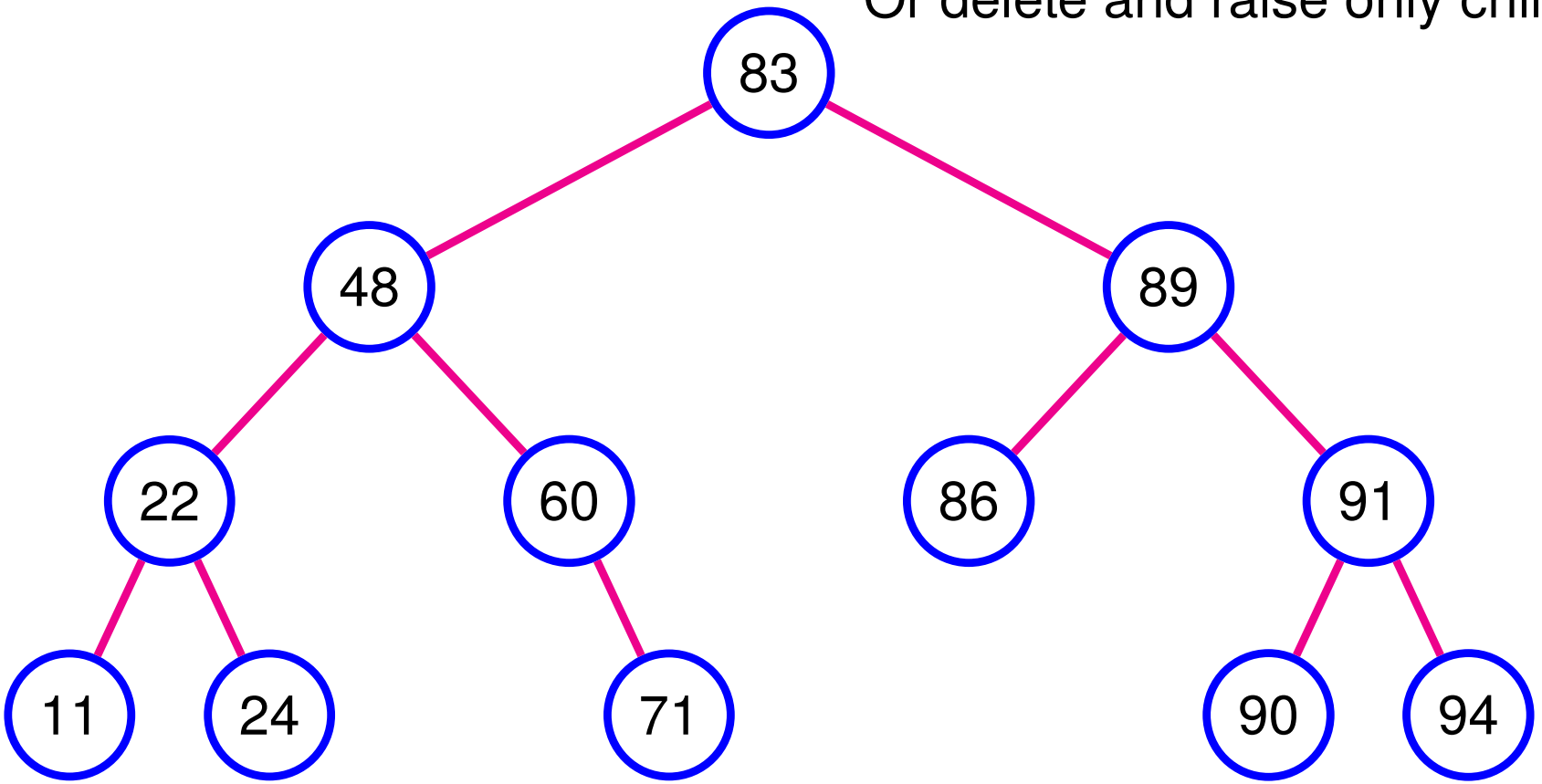
BST delete

For internal nodes: find “next” node.

I.e., leftmost desc. of right child.

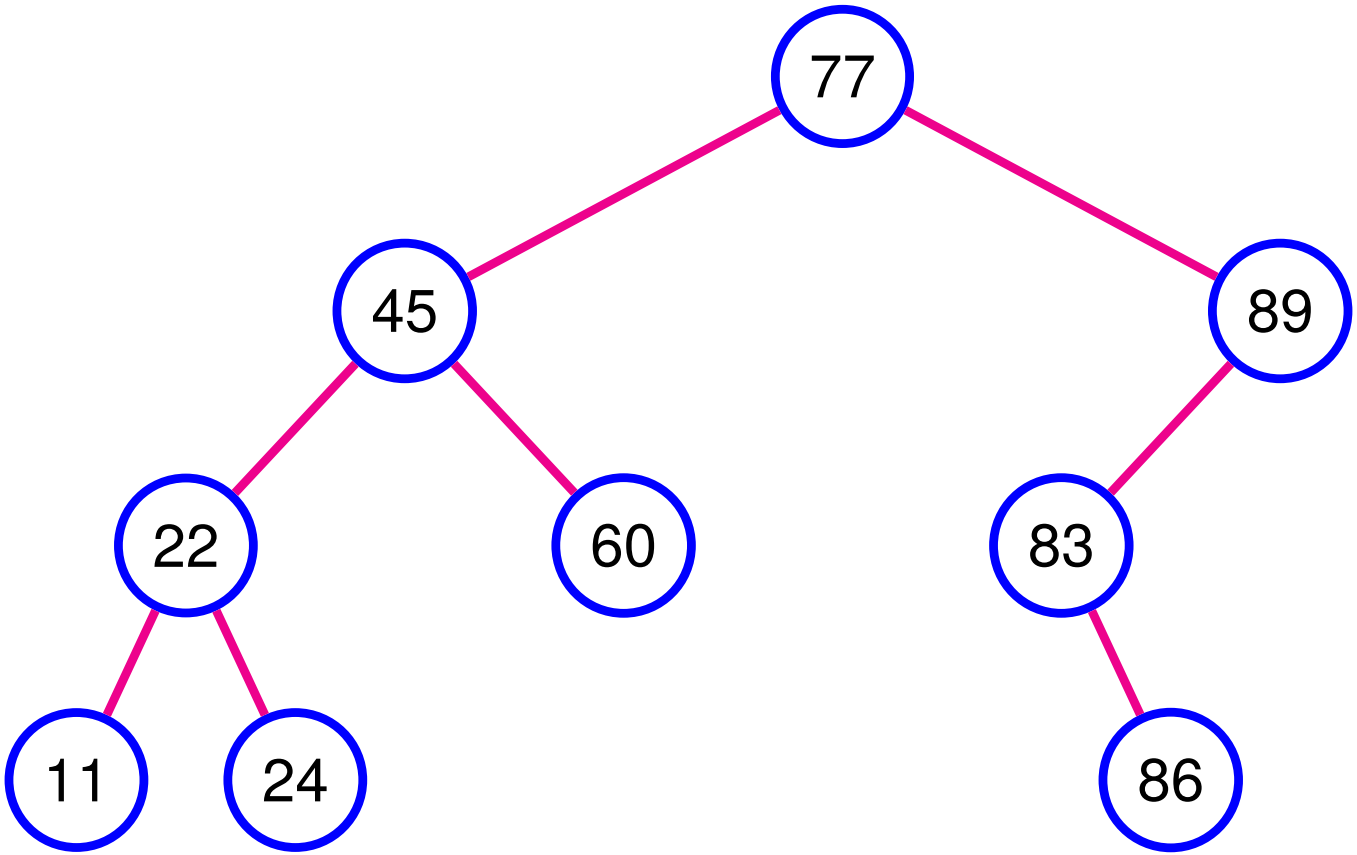
Swap them, then delete leaf.

Or delete and raise only child.



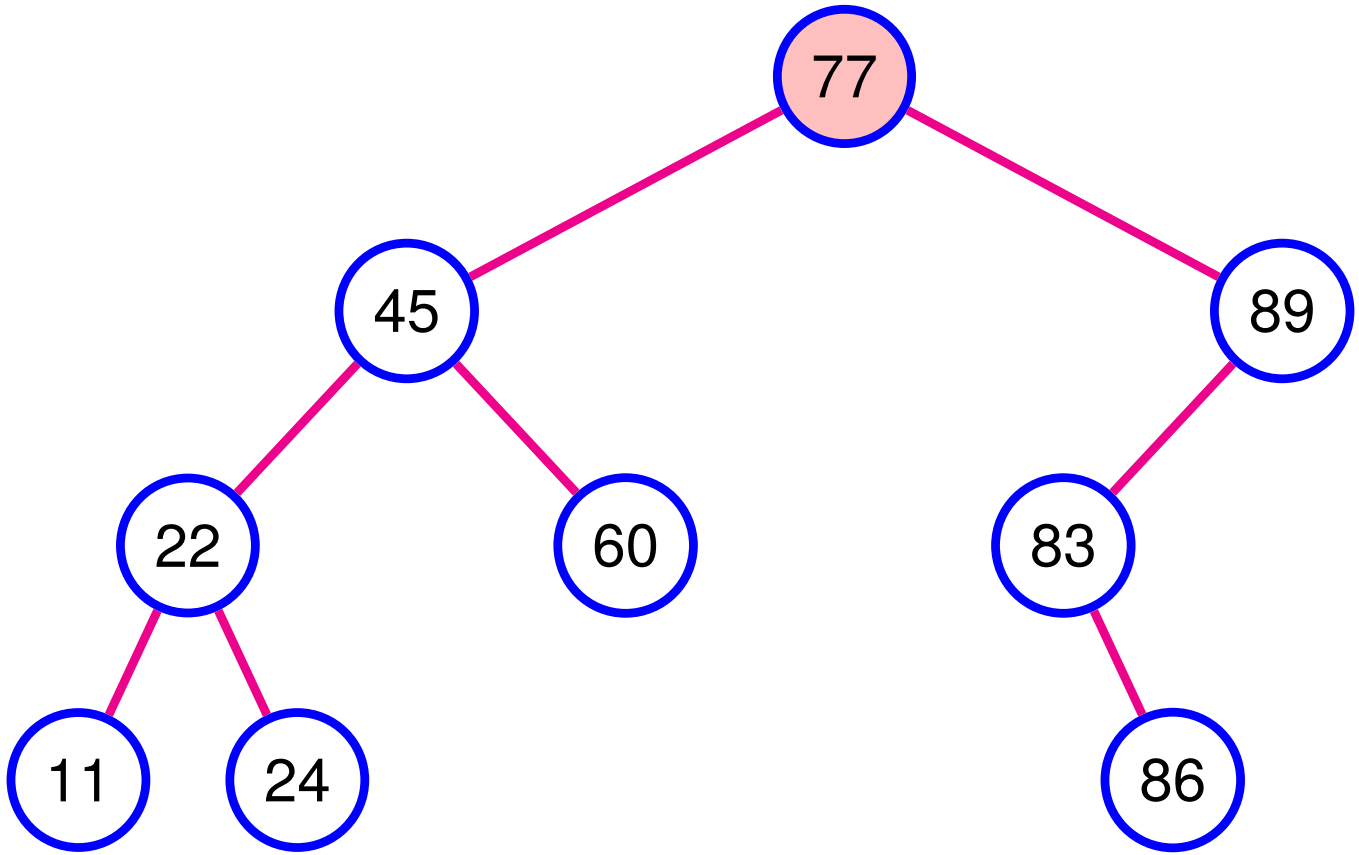
BST insert

Let's insert 71.



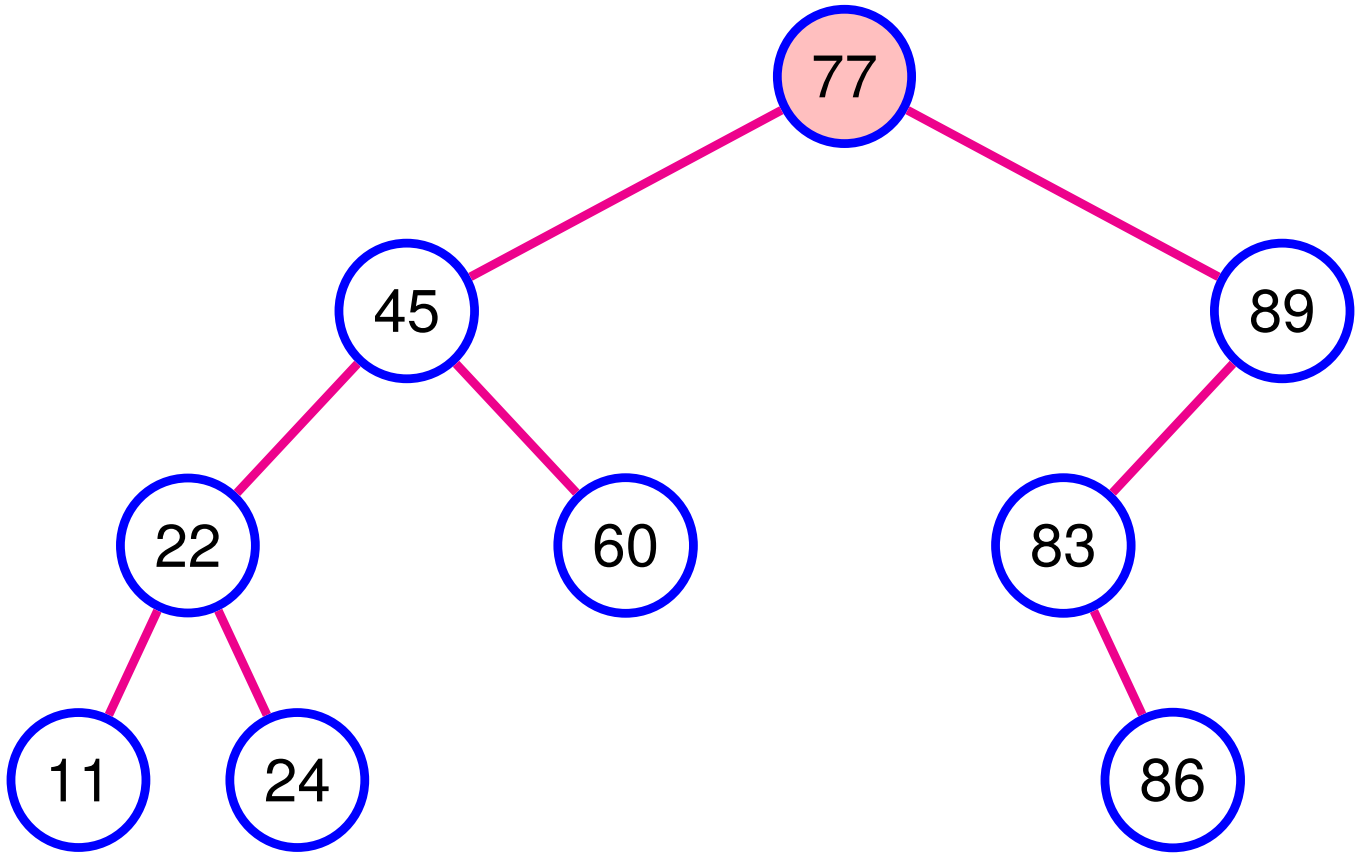
BST insert

Let's insert 71.
Less than 77, insert into ???



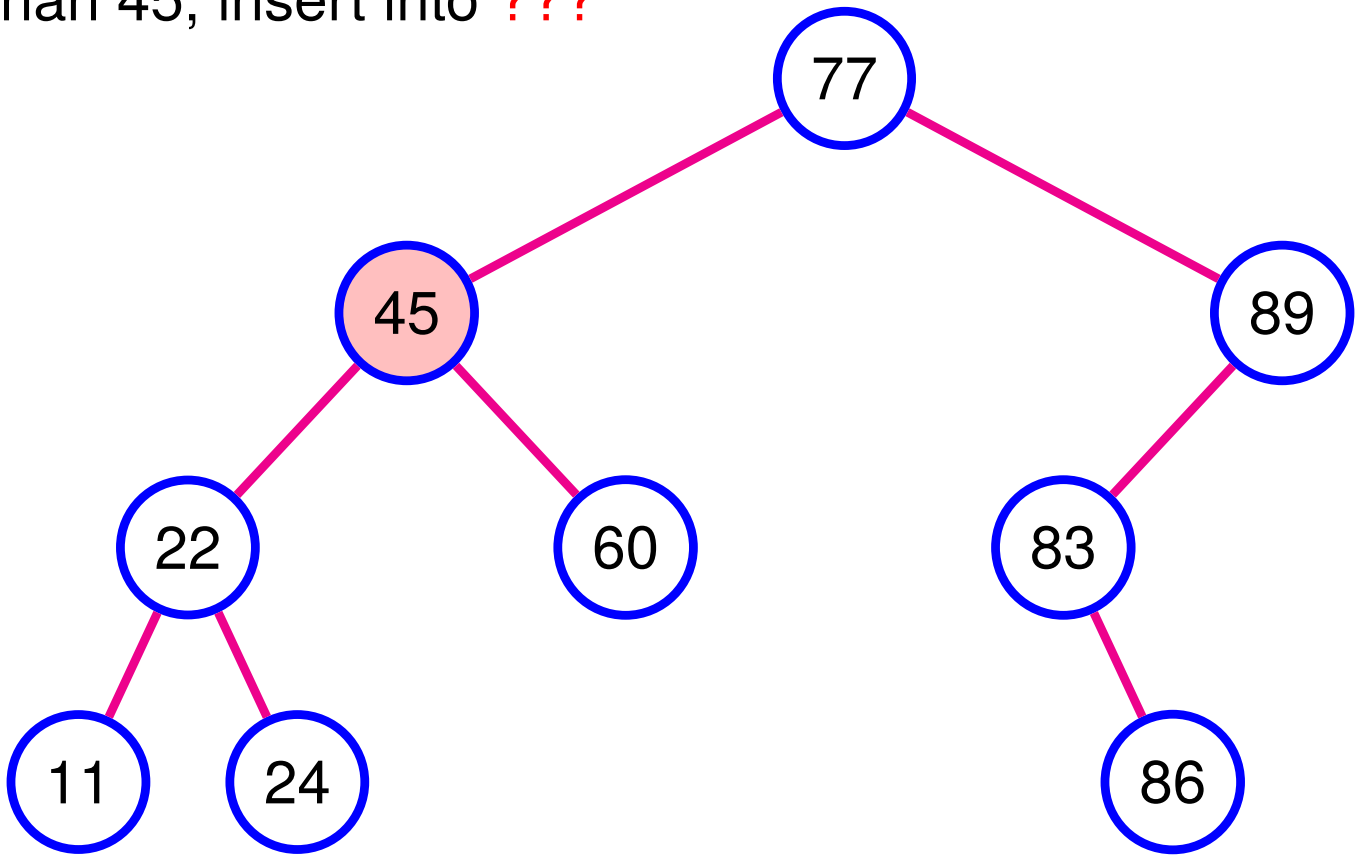
BST insert

Let's insert 71.
Less than 77, insert into left.



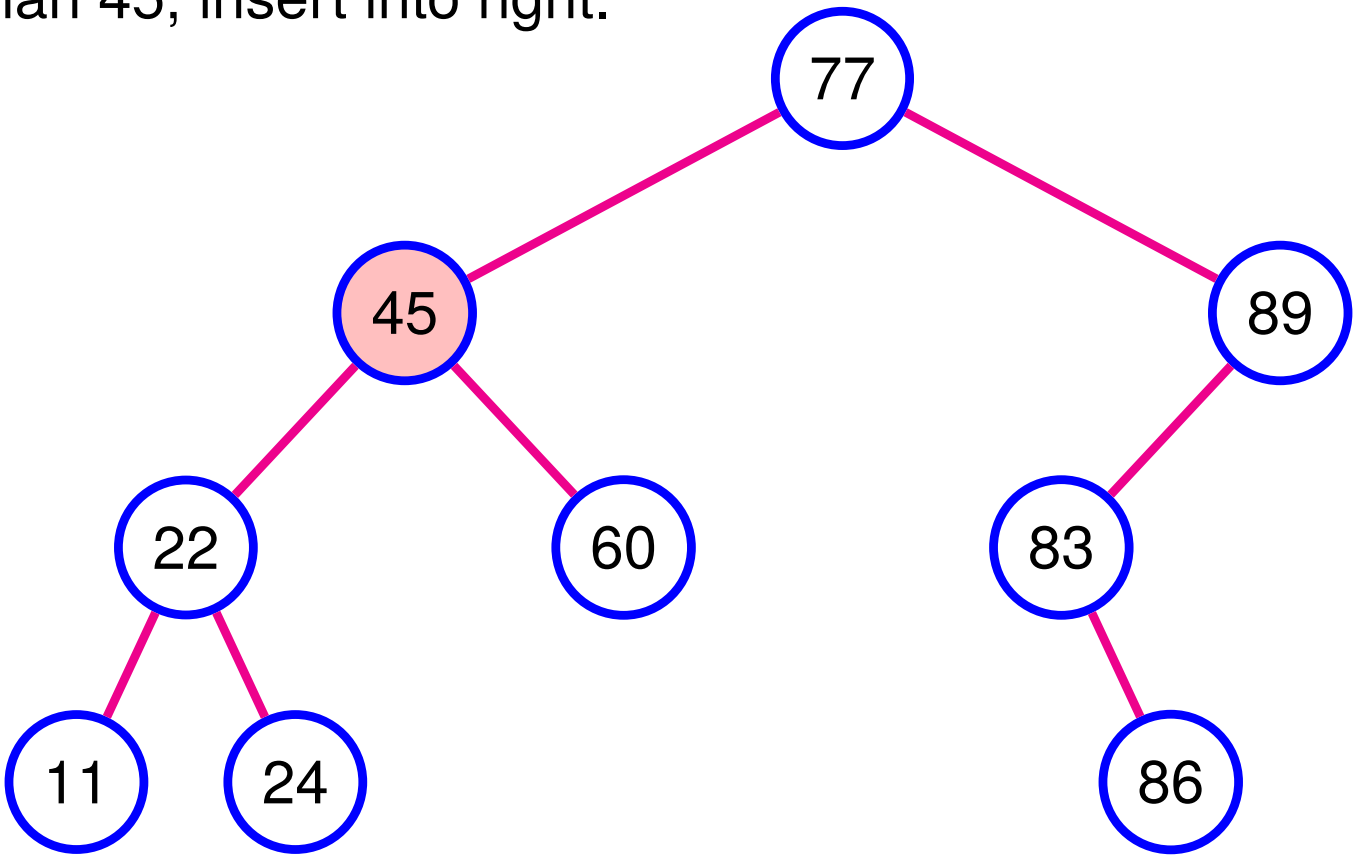
BST insert

Let's insert 71.
Less than 77, insert into left.
Greater than 45, insert into ???



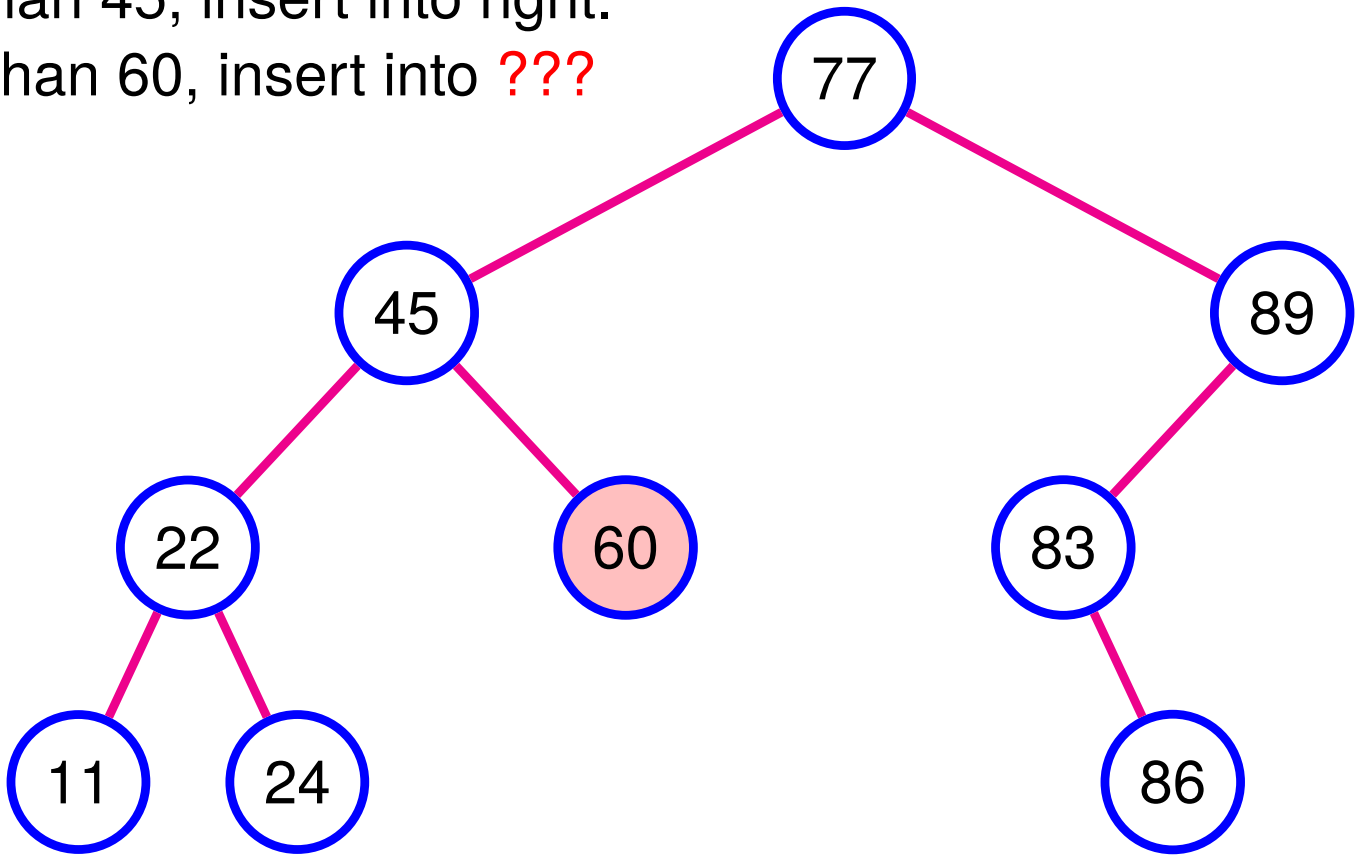
BST insert

Let's insert 71.
Less than 77, insert into left.
Greater than 45, insert into right.



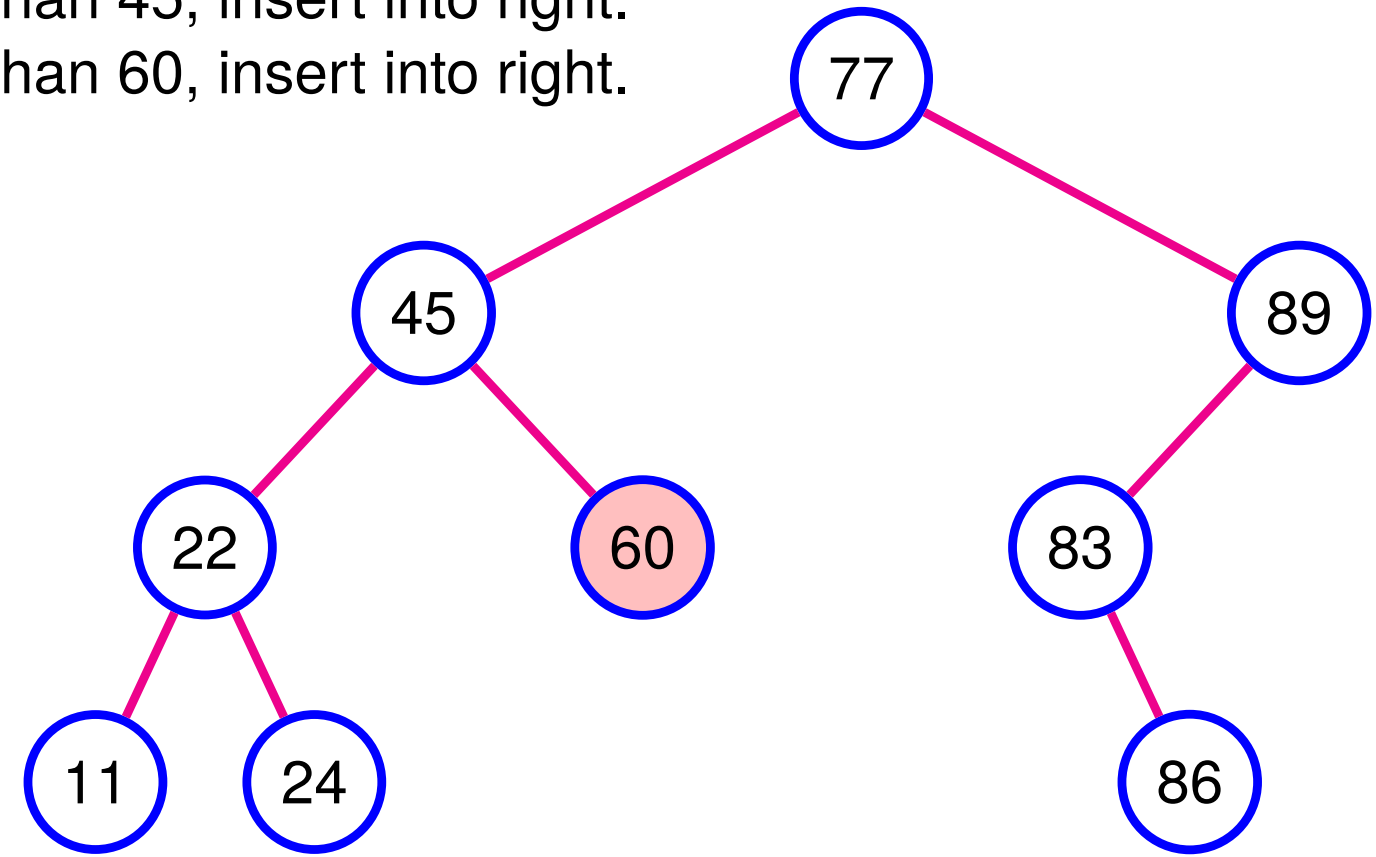
BST insert

Let's insert 71.
Less than 77, insert into left.
Greater than 45, insert into right.
Greater than 60, insert into ???



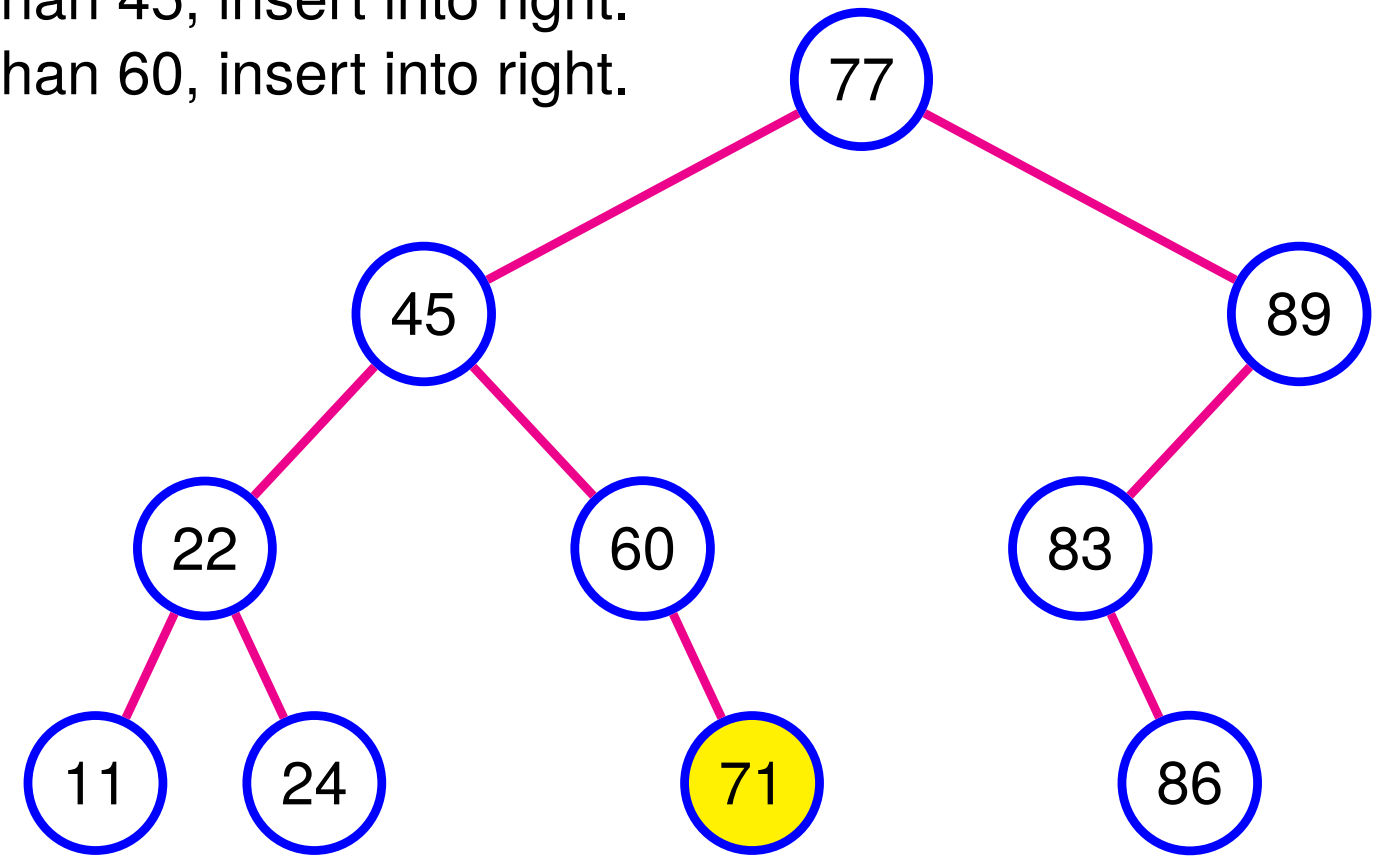
BST insert

Let's insert 71.
Less than 77, insert into left.
Greater than 45, insert into right.
Greater than 60, insert into right.



BST insert

Let's insert 71.
Less than 77, insert into left.
Greater than 45, insert into right.
Greater than 60, insert into right.



BST insert algorithm

```
Function put(node, key, value) is
    Output: the updated node

    if node is None then
        return new node with key, value, and two None children
    else if key < node.key then
        node.left ← put(node.left, key, value);
        return node
    else if key > node.key then
        node.right ← put(node.right, key, value);
        return node
    else
        node.val = value;
        return node
    end
end
```

Leaf insertion: find where the key *would* go, then add it as a leaf at that position.

BST insert algorithm

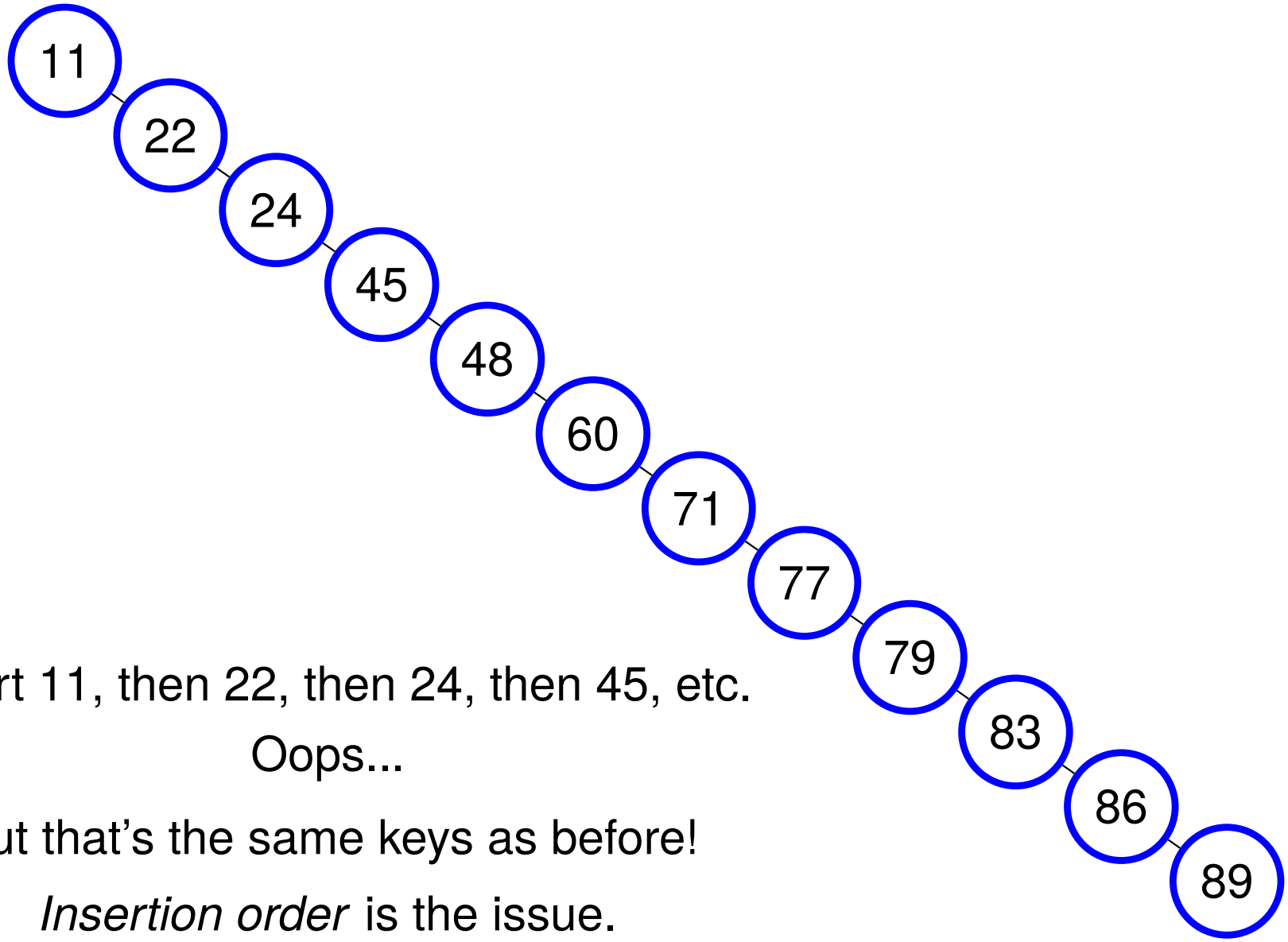
```
Function put(node, key, value) is
    Output: the updated node

    if node is None then
        return new node with key, value, and two None children
    else if key < node.key then
        node.left ← put(node.left, key, value);
        return node
    else if key > node.key then
        node.right ← put(node.right, key, value);
        return node
    else
        node.val = value;
        return node
    end
end
```

Leaf insertion: find where the key *would* go, then add it as a leaf at that position.

- *Always* as a leaf… What could possibly go wrong?

BST insert: Pathological case



Insert 11, then 22, then 24, then 45, etc.

Oops...

But that's the same keys as before!

Insertion order is the issue.

We're in trouble

Both insertion and lookup traverse from the root to a leaf (in the worst case).

These are $\mathcal{O}(\mathit{height})$ operations.

- They depend on how *tall* the tree is.

If tree is balanced, $\mathit{height} = \mathcal{O}(\log n)$

- We had that for binary heaps (complete binary trees)

If tree is completely unbalanced, $\mathit{height} = \mathcal{O}(n)$

- Our tree is really just a linked list!

n	$\lceil \log_2 n \rceil$
10	4
1,000	10
1,000,000	20
1,000,000,000	30

The Solution: Self-Balancing BSTs

- BSTs that are guaranteed to be *almost* balanced
  - Not *perfectly* balanced, but within a constant factor
    - E.g., no path is more than 2$\times$ as long as another
  - That's enough to get $\mathcal{O}(\log n)$ (constants don't matter)
  - And perfect balance is too hard/expensive to maintain

The Solution: Self-Balancing BSTs

- BSTs that are guaranteed to be *almost* balanced
  - Not *perfectly* balanced, but within a constant factor
    - E.g., no path is more than 2× as long as another
  - That's enough to get $\mathcal{O}(\log n)$ (constants don't matter)
  - And perfect balance is too hard/expensive to maintain

- Very attractive data structures problem!
  - Who doesn't love dictionaries with $\mathcal{O}(\log n)$ operations? (Worst case!)

- Lots of solutions, starting in the 60s
  - Lots of improvements over the decades
  - Each building on top of previous ones
    - And getting more and more complex
    - Can get pretty hairy! (some I have to relearn every time)
  - Could have a whole quarter on those! (But not worth it)

Today: The first (and simplest) solution: *AVL Trees* (1962)

Stop! Question time!

Before we move on to how self-balancing BSTs work...

Anything unclear so far?

Anything I missed?

Self-Balancing Trees Overview

Self-Balancing Binary Search Trees

Observation: insertions (and deletions) are what may introduce unbalance (i.e., not lookups, and not updates)

Self-Balancing Binary Search Trees

Observation: insertions (and deletions) are what may introduce unbalance (i.e., not lookups, and not updates)

Strategy: Insert (or delete) as before, *then*:

- Use *invariants* to detect unbalance
  - Broken invariant $\rightarrow$ tree is unbalanced
- Perform *repairs* to restore balance after insertion/deletion
  - Shuffle nodes around, but preserve BST order
- **Key**: fix as we go along, don't let things get out of hand
  - So we only have small unbalance to fix each time

Think Globally, Act Locally

- Balance is a *global* property: about the entire tree
- But we must detect unbalance *locally*
- And we must repair unbalance with *local* changes

Think Globally, Act Locally

- Balance is a *global* property: about the entire tree
- But we must detect unbalance *locally*
- And we must repair unbalance with *local* changes

- Why?
  - Looking at the entire tree, could figure out optimal shape
  - But entire tree = $\mathcal{O}(n)$! No go!
    - Want: $\mathcal{O}(\log n)$ *worst case*!
  - So both detection and repairs bounded by $\mathcal{O}(\log n)$
    - $\mathcal{O}(\log n)$ = one path from root to leaves

Think Globally, Act Locally

- Balance is a *global* property: about the entire tree
- But we must detect unbalance *locally*
- And we must repair unbalance with *local* changes

- Why?
  - Looking at the entire tree, could figure out optimal shape
  - But entire tree = $\mathcal{O}(n)$! No go!
    - Want: $\mathcal{O}(\log n)$ *worst case*!
  - So both detection and repairs bounded by $\mathcal{O}(\log n)$
    - $\mathcal{O}(\log n)$ = one path from root to leaves

- Good news: insertion/deletion are *local* operations!
  - Any unbalance they introduce is *local*!
    - E.g., insert into left subtree, right subtree is unchanged. (same thing we saw with binary heaps)
  - And *bounded*: can only increase/decrease height by one!
  - So we can detect and repair locally

AVL trees

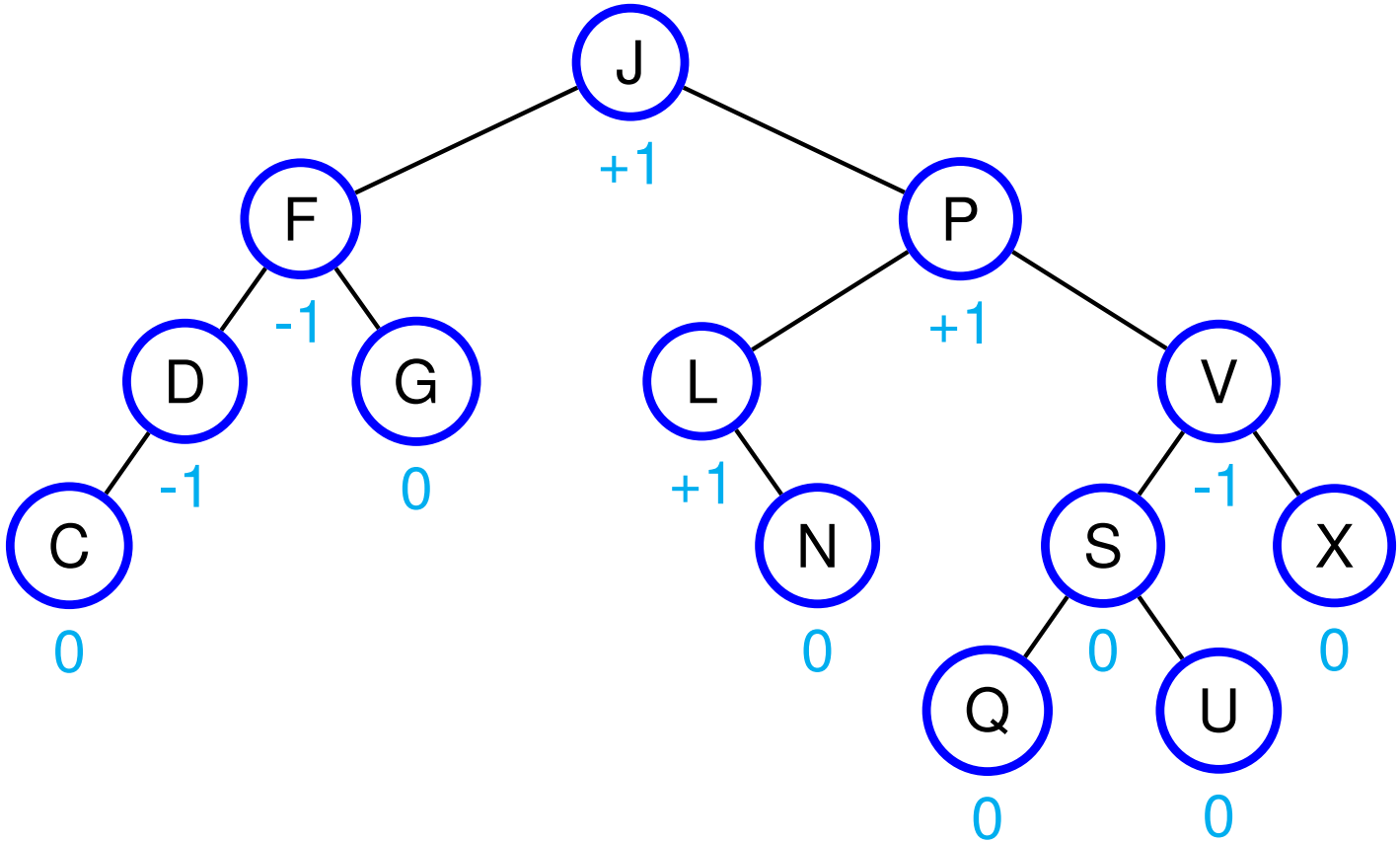
Due to Georgy ***A***delson-***V***elsky & Evgenii ***L***andis, in 1962

- The first solution to the problem of self-balancing trees
- A good entry point for us to understand them!

Each node stores a *balance factor*, which is the difference between the height of its right subtree and its left one.

- **Invariant**: each balance factor must be between -1 and +1
  - If an insertion/deletion ever causes a node to go outside that range, we have detected unbalance!
- **Repair**: series of rotations
  - Depends on which way the unbalance goes (-2 or +2)

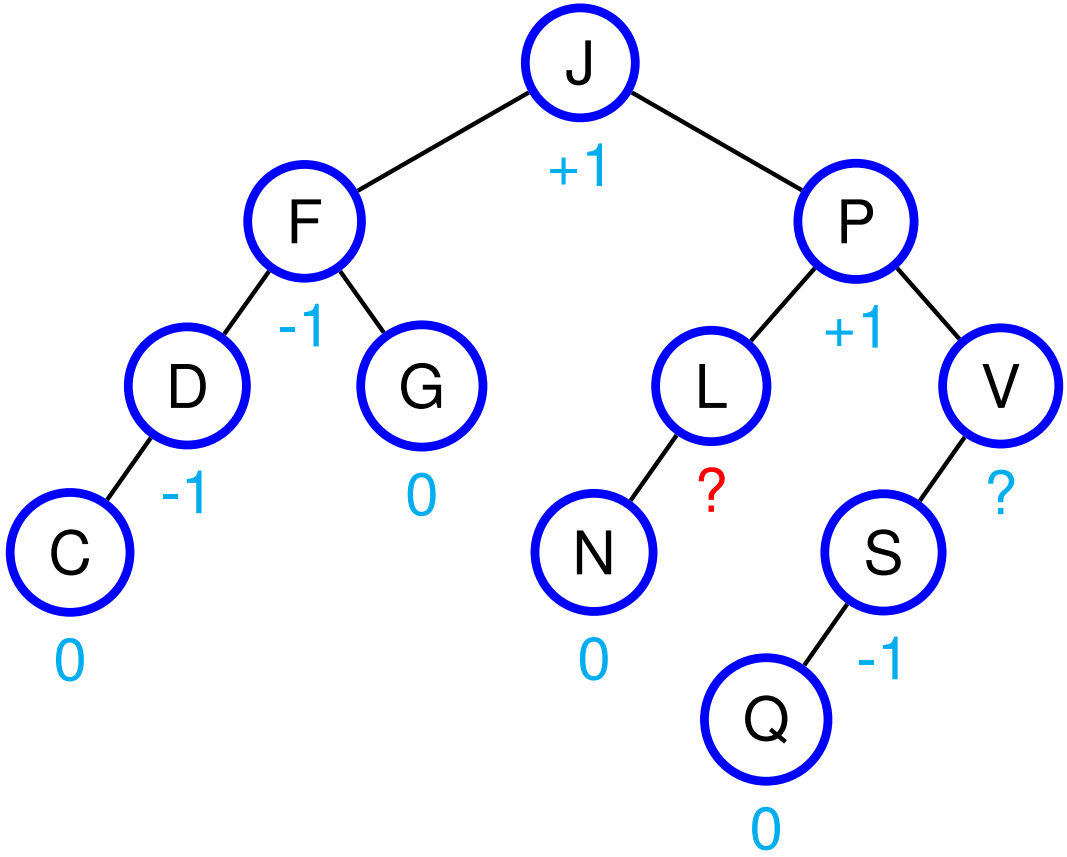
Invariant: Balance factors



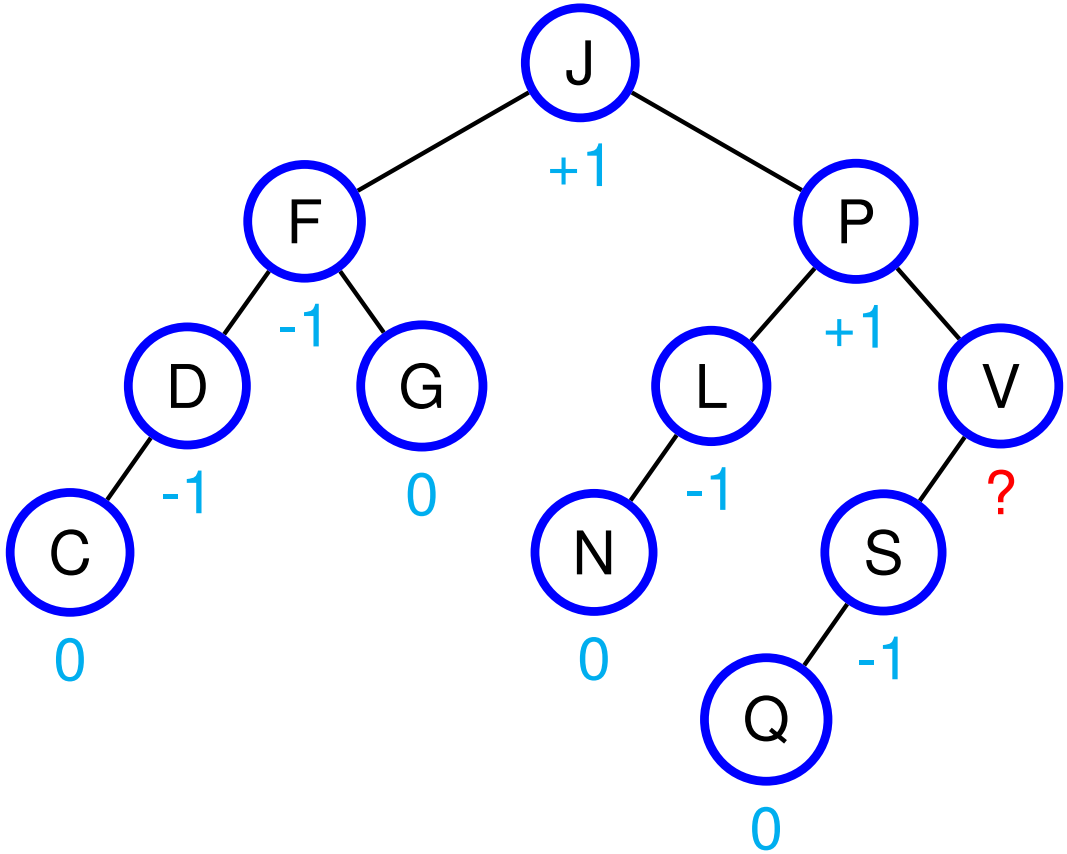
-1: left subtree taller by 1
0: both subtrees equally tall
+1: right subtree taller by 1

Invariant: balance factors must stay in the range $[-1, +1]$

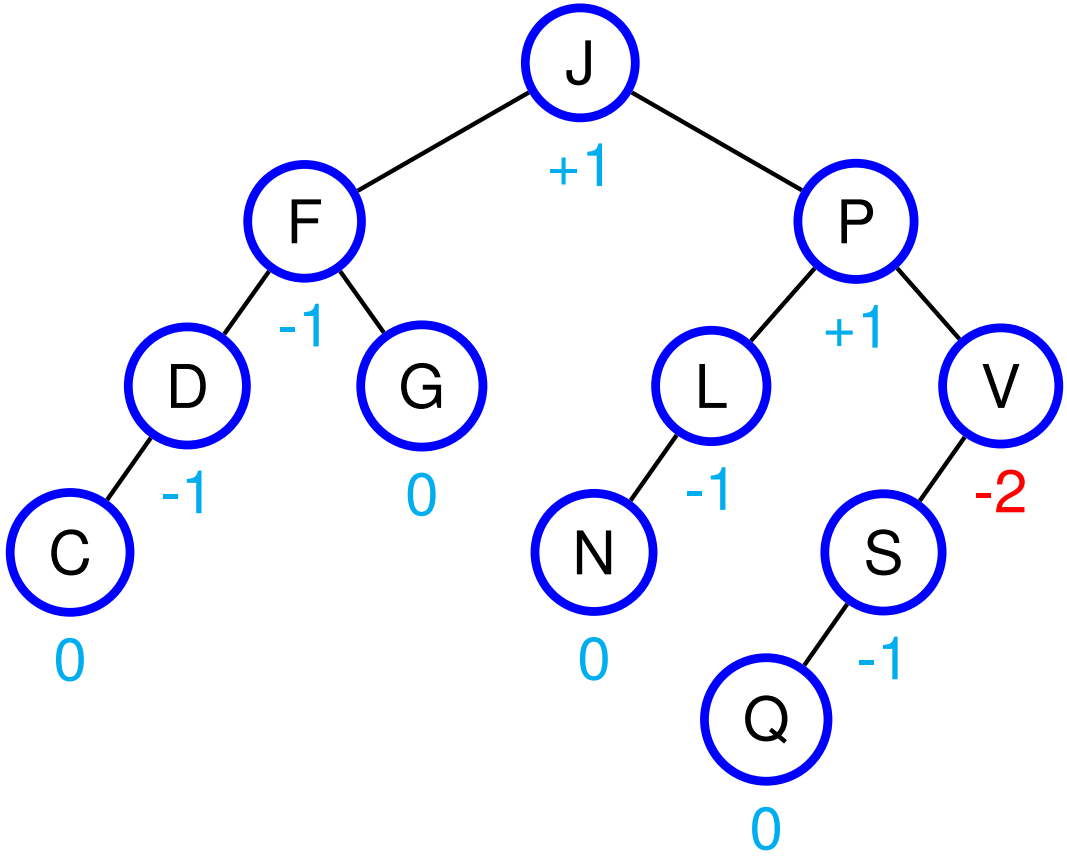
Invariant: Balance factors



Invariant: Balance factors



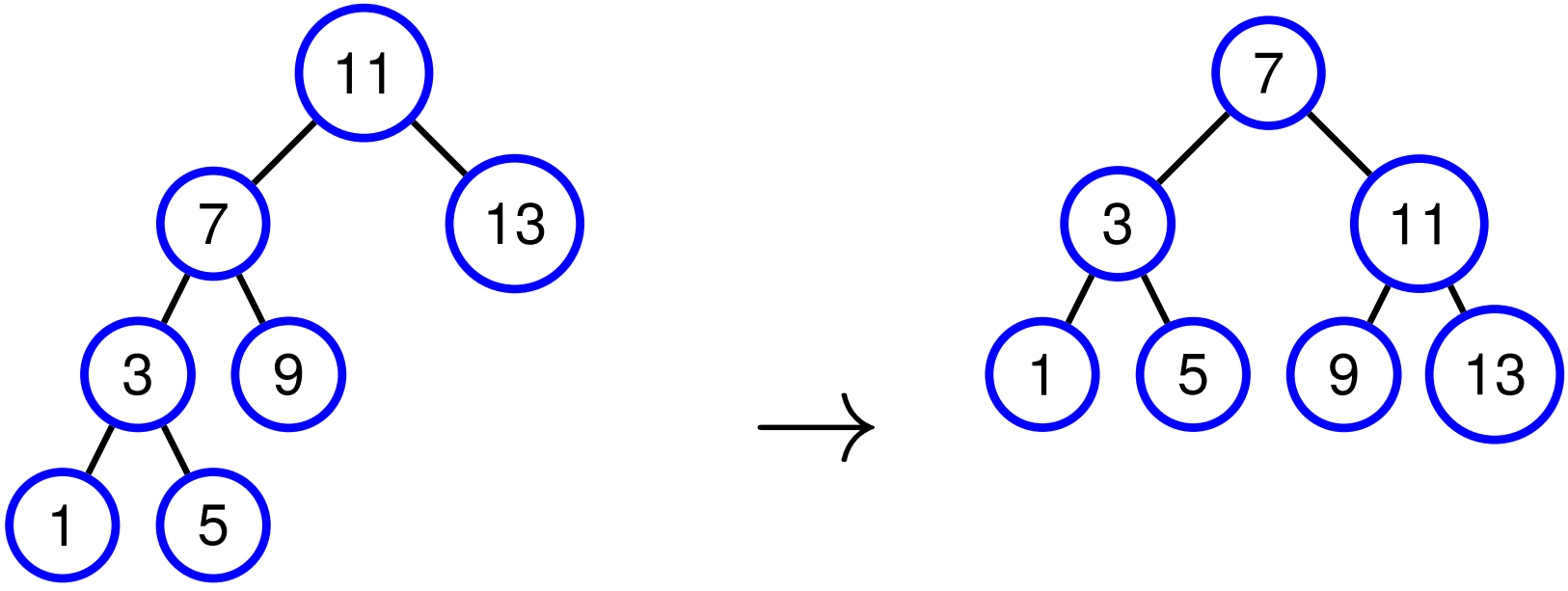
Invariant: Balance factors



V has a balance factor of -2. Its left subtree is too tall.

Invariant is broken! $\rightarrow$ V subtree is too far out of balance!

Repair: Tree rotations

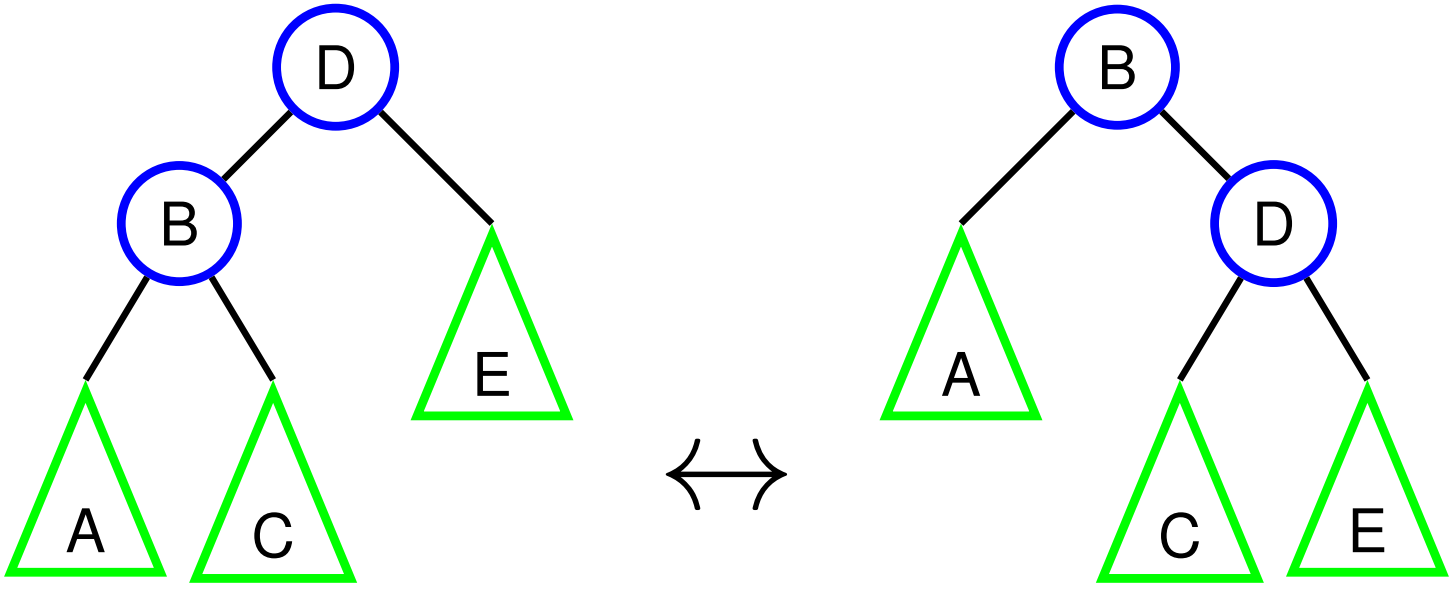


Tree on the left is unbalanced, tree on the right is balanced.

But same elements, and BST invariant respected for both!

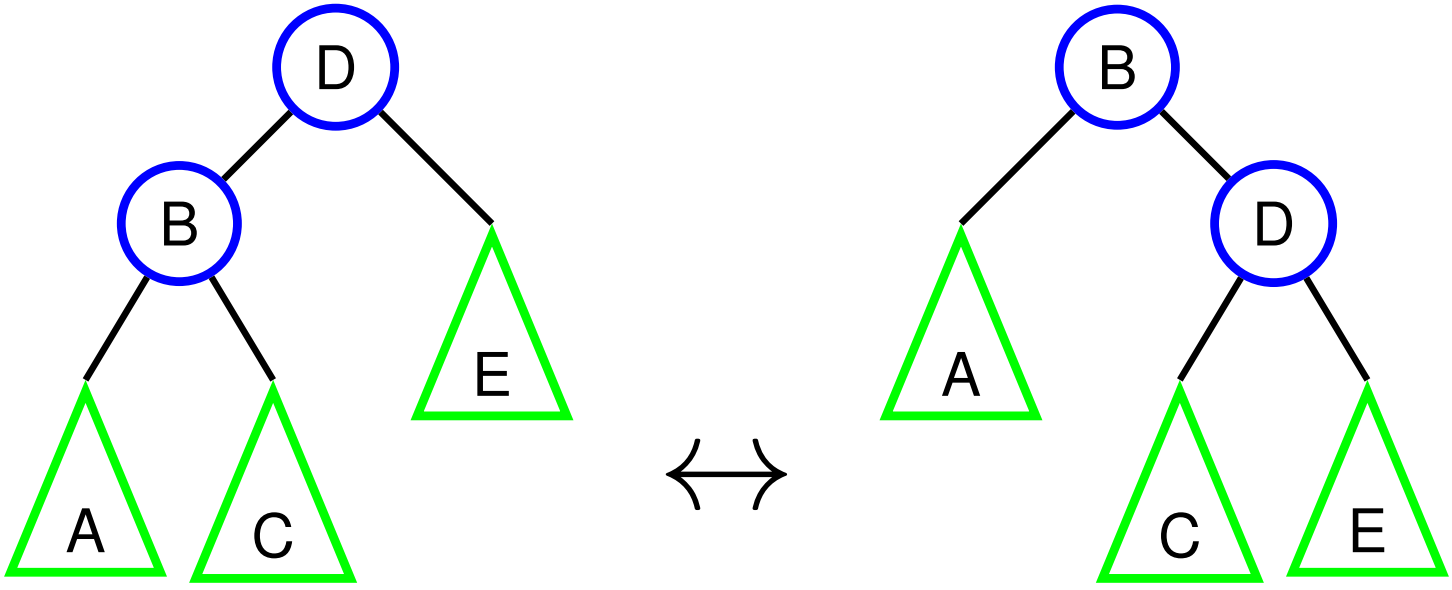
Can “rotate to the right” centered around 11 to go from L to R.

Repair: Tree rotations



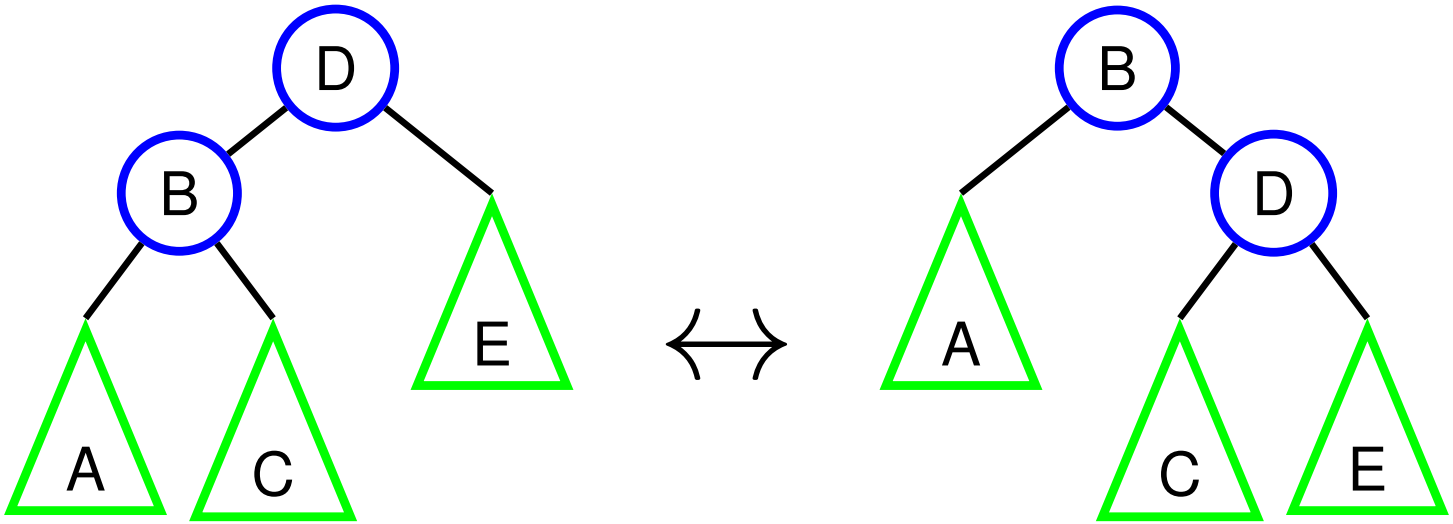
- Triangles (A, C, E) represent arbitrary subtrees
  - But A on the left is the same subtree as A on the right
- Both trees have all the same keys!
- And both trees respect the BST invariant!
- But one may be more balanced than the other!

Repair: Tree rotations



- Going left-to-right: right rotation.
- Going right-to-left: left rotation.
- As with relaxation and SSSP, *when*, *where*, and *how* to rotate will be decided by the specific algorithms we use.

Repair: Tree rotations




```
def rotate_right(d):
    let b = d.left
    d.left = b.right
    b.right = d
    return b
```

```
def rotate_left(b):
    let d = b.right
    b.right = d.left
    d.left = b
    return d
```

It's $\mathcal{O}(1)$ work!

Stop! Question time!

Before we move on to how to insert into AVL trees...

Anything unclear so far?

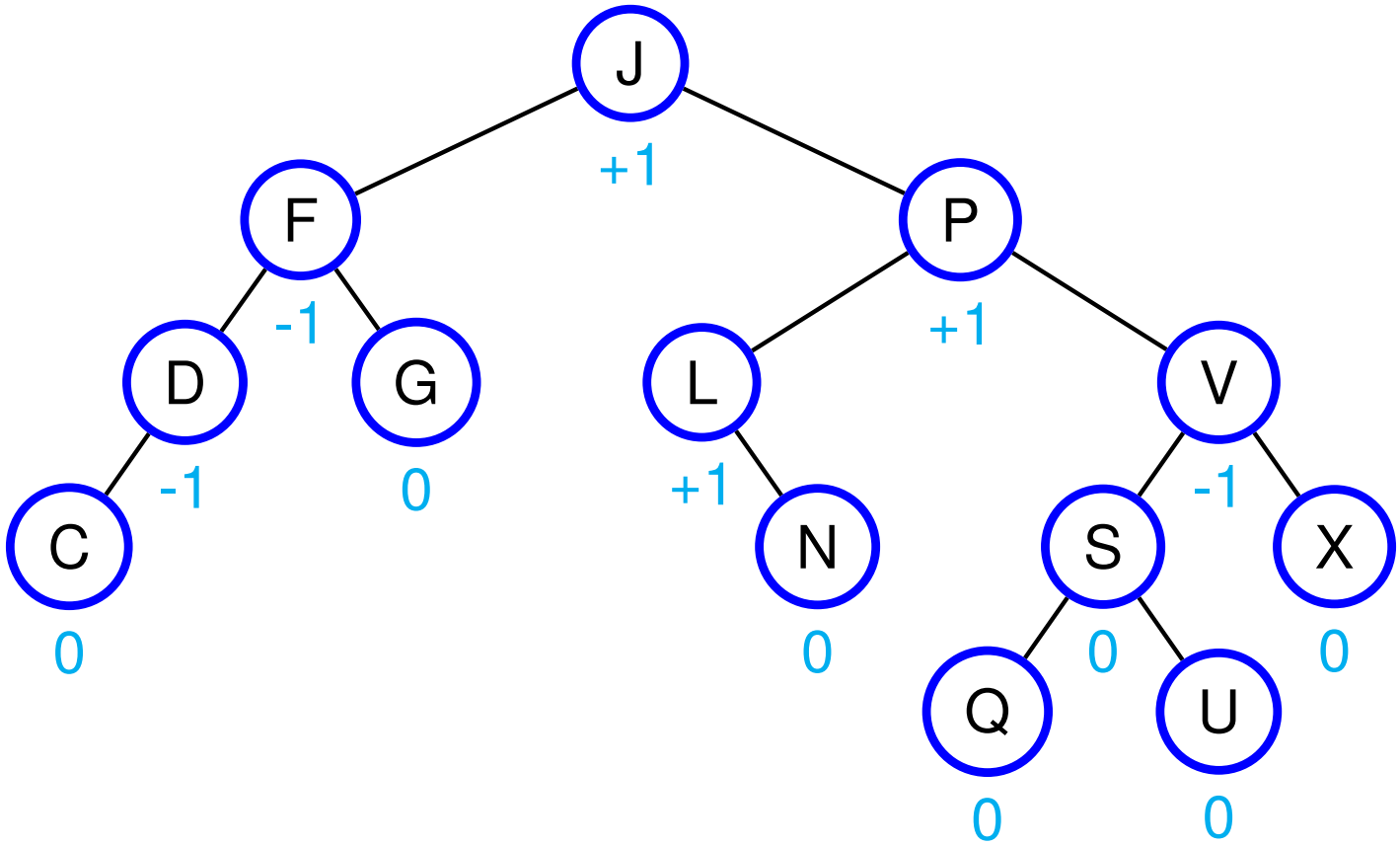
Anything I missed?

AVL insertion

1. First do a normal leaf insertion (recursively)
2. At each step, insertion returns whether the height of the subtree increased
   - Insertion can only either increase height, or keep the same
3. Based on that, adjust balance factors on the way back up (while returning from the recursive calls)
4. If the balance factor is outside the range $[-1, +1]$, rotate to restore balance
5. Keep returning all the way up to the root

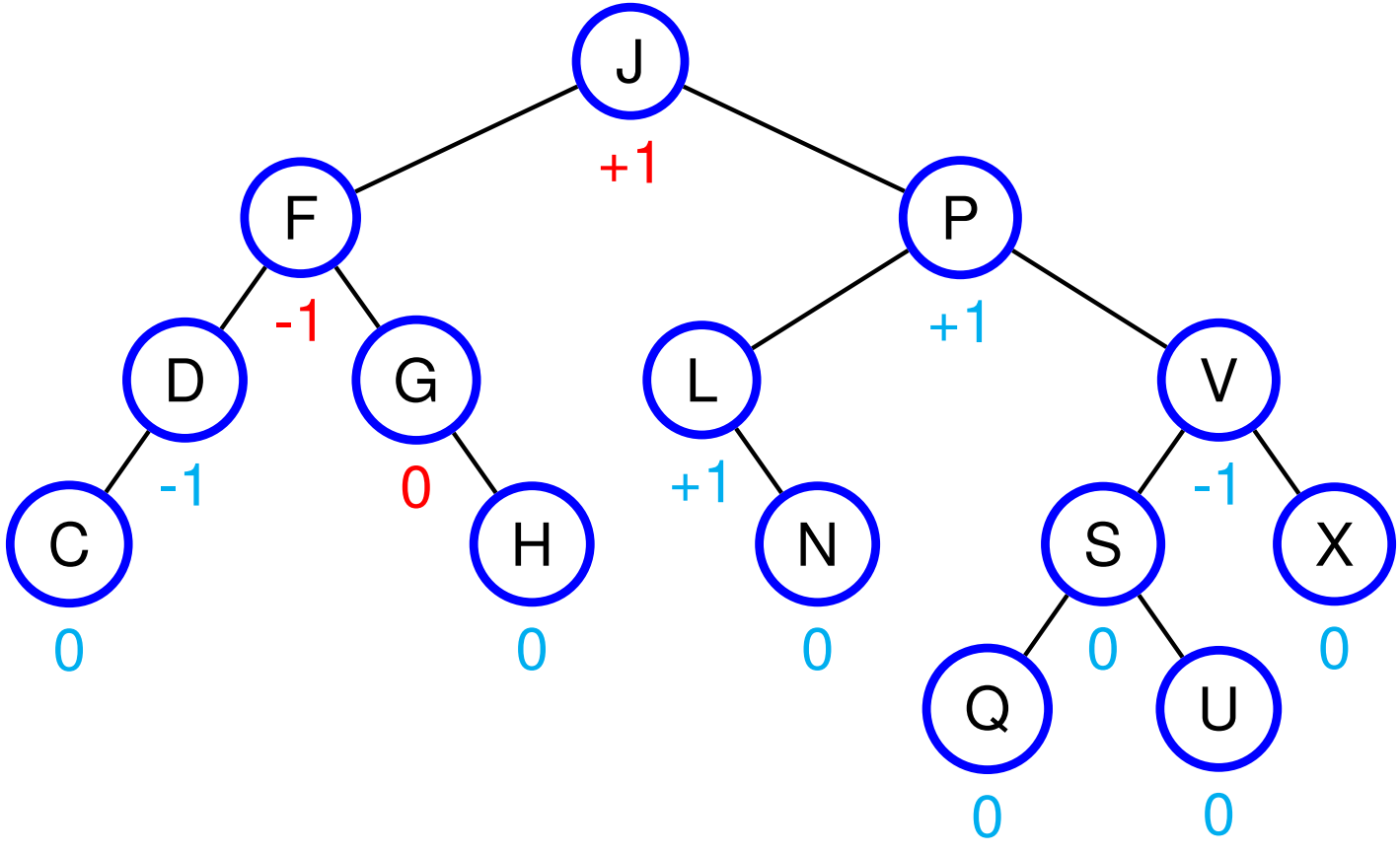
AVL insertion example

Let's insert H:



AVL insertion example

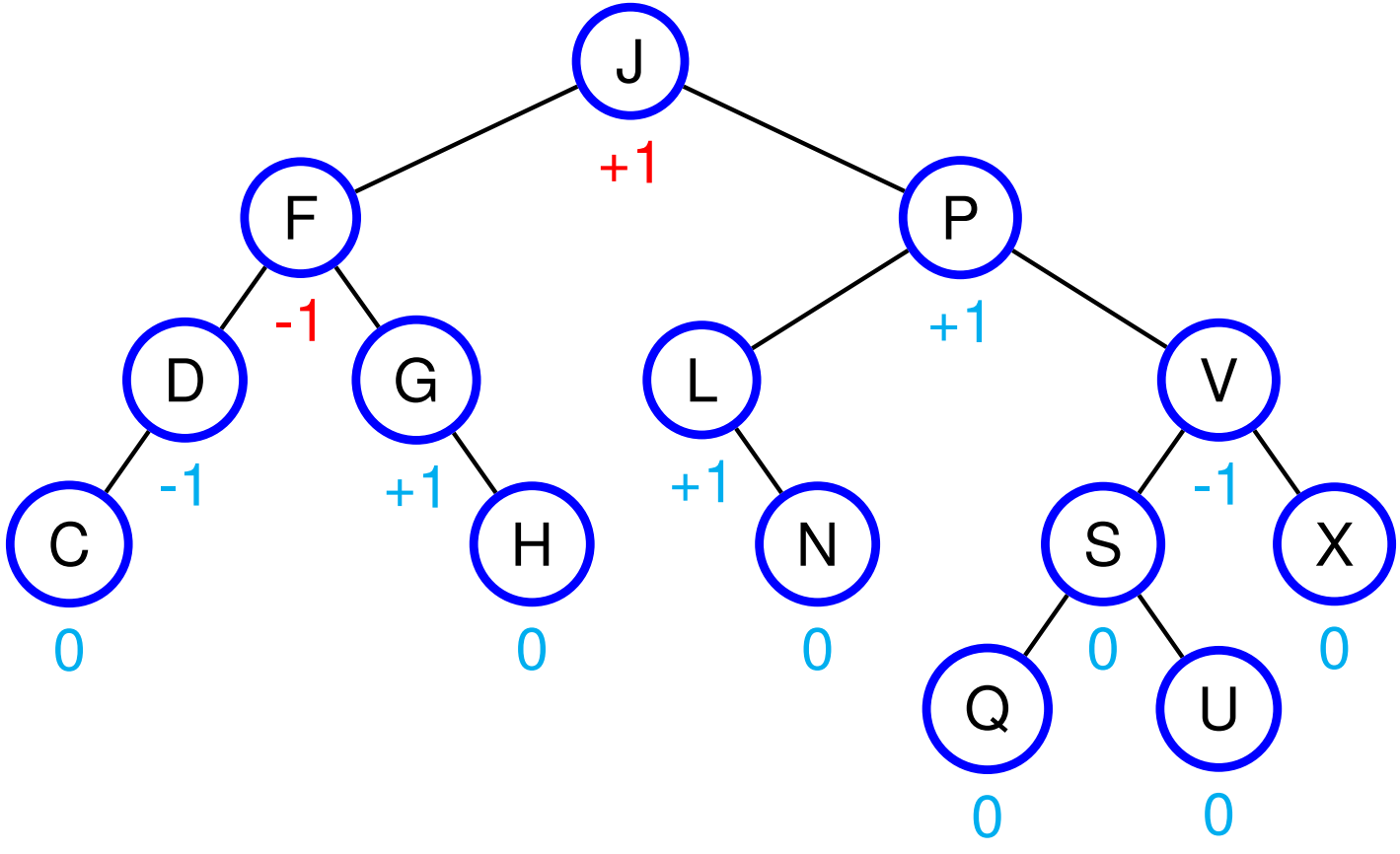
Let's insert H:



Leaf insert, invalidating balance factors along path.

AVL insertion example

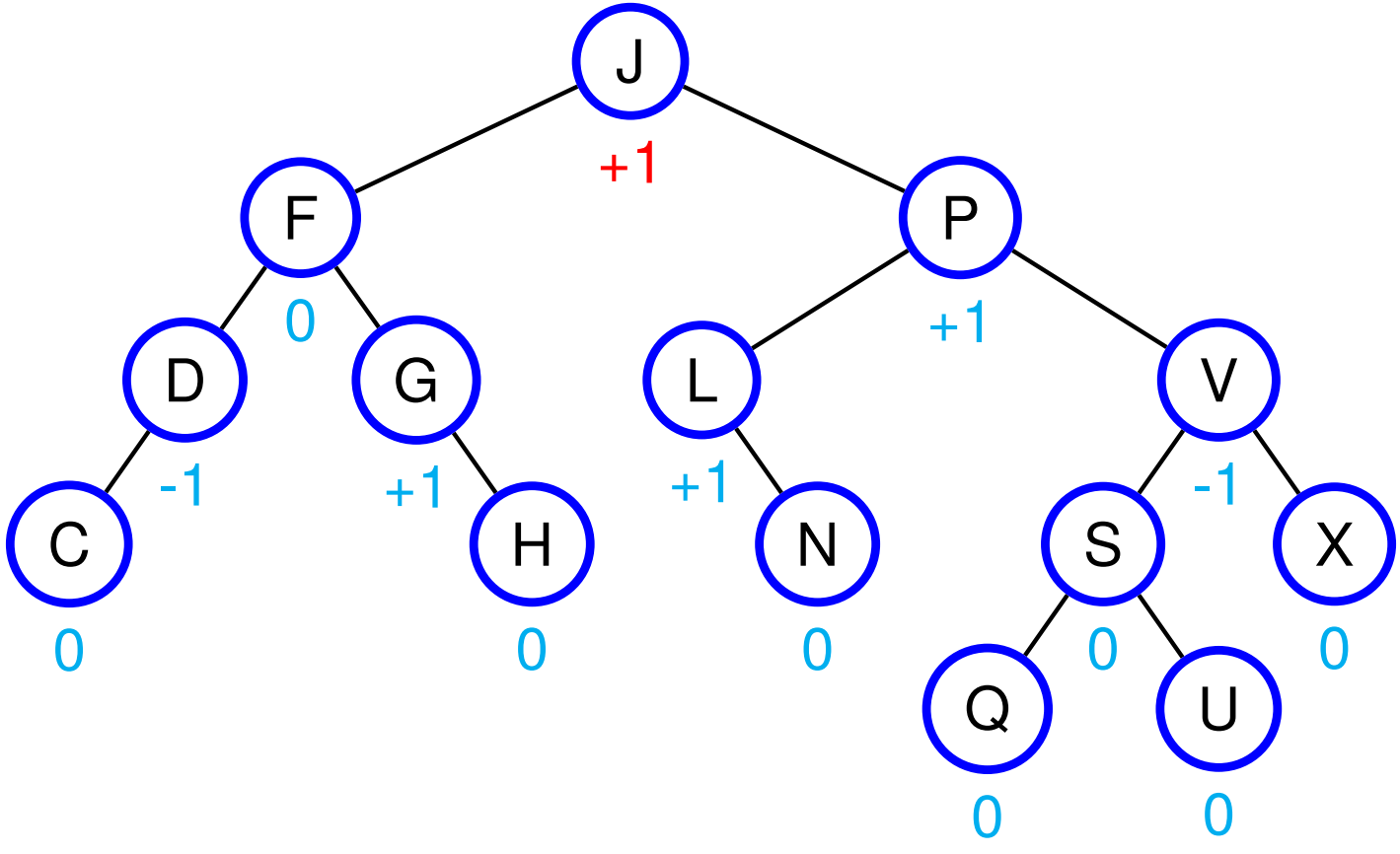
Let's insert H:



Right subtree got taller, adjust balance. Still $[-1, +1]$; we're fine.

AVL insertion example

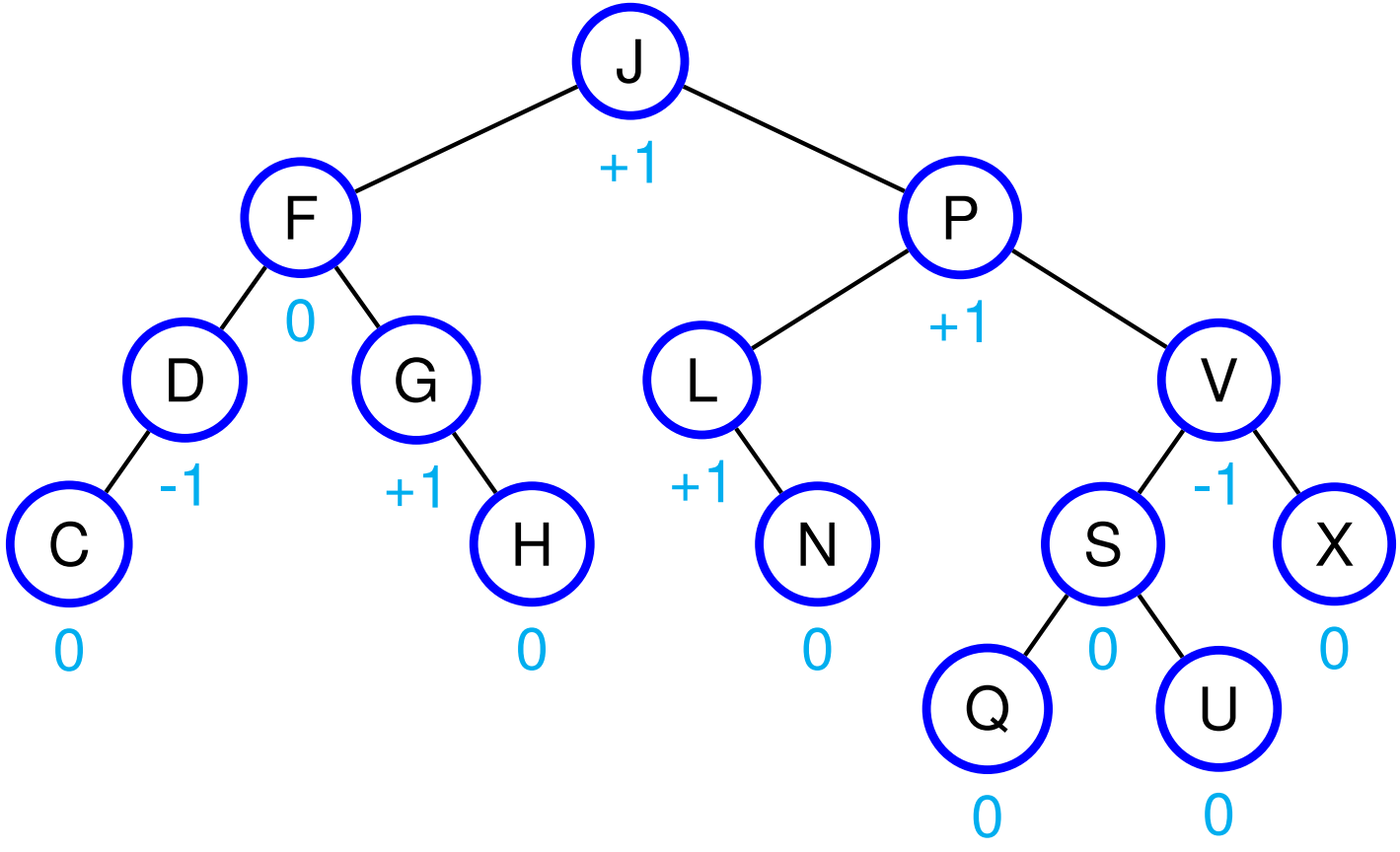
Let's insert H:



Right subtree of F also got taller. Still $[-1, +1]$; we're fine.

AVL insertion example

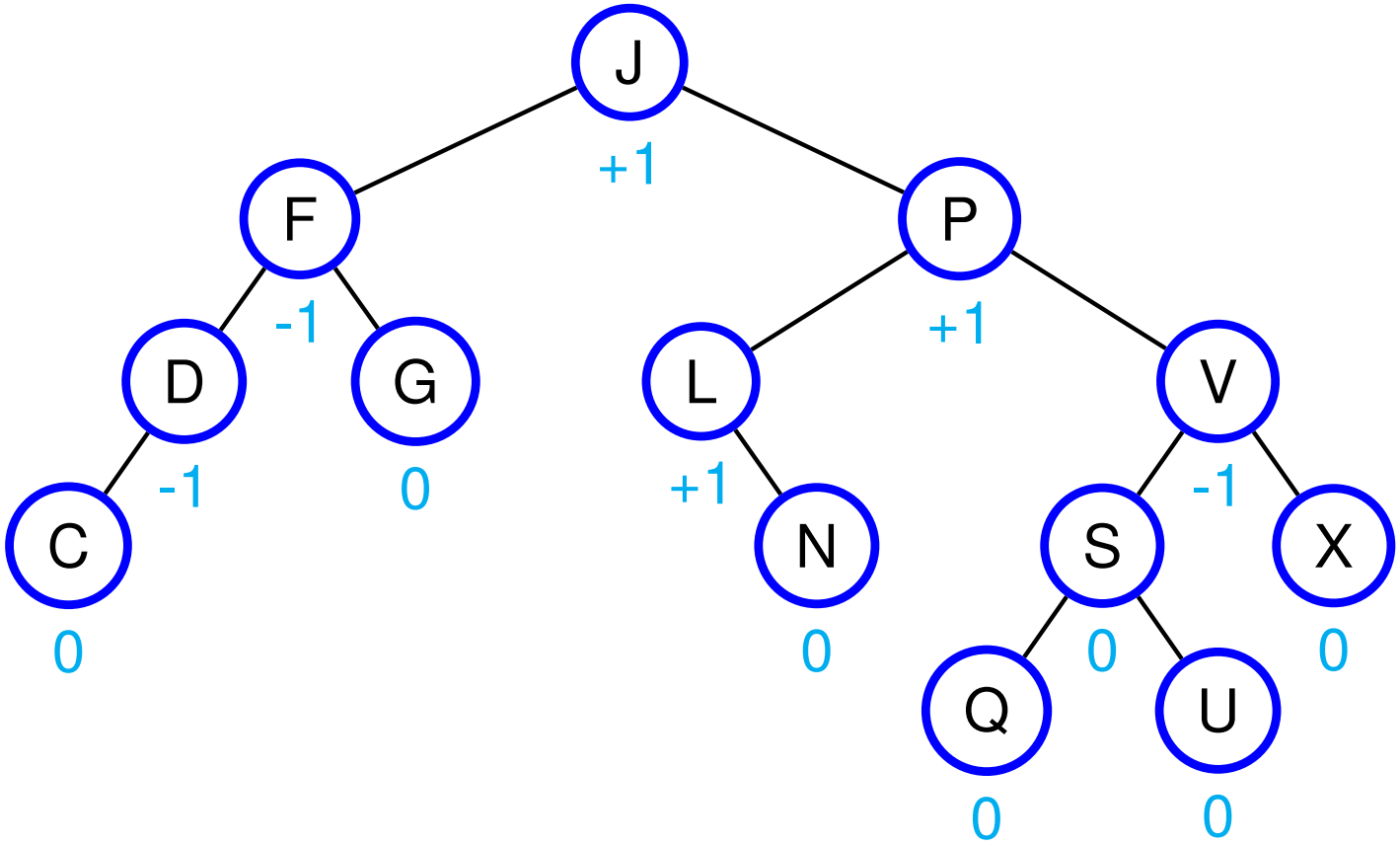
Let's insert H:



Left subtree of F was the taller one, though.
So overall subtree at F did not grow.
No need to continue further up. We're done.

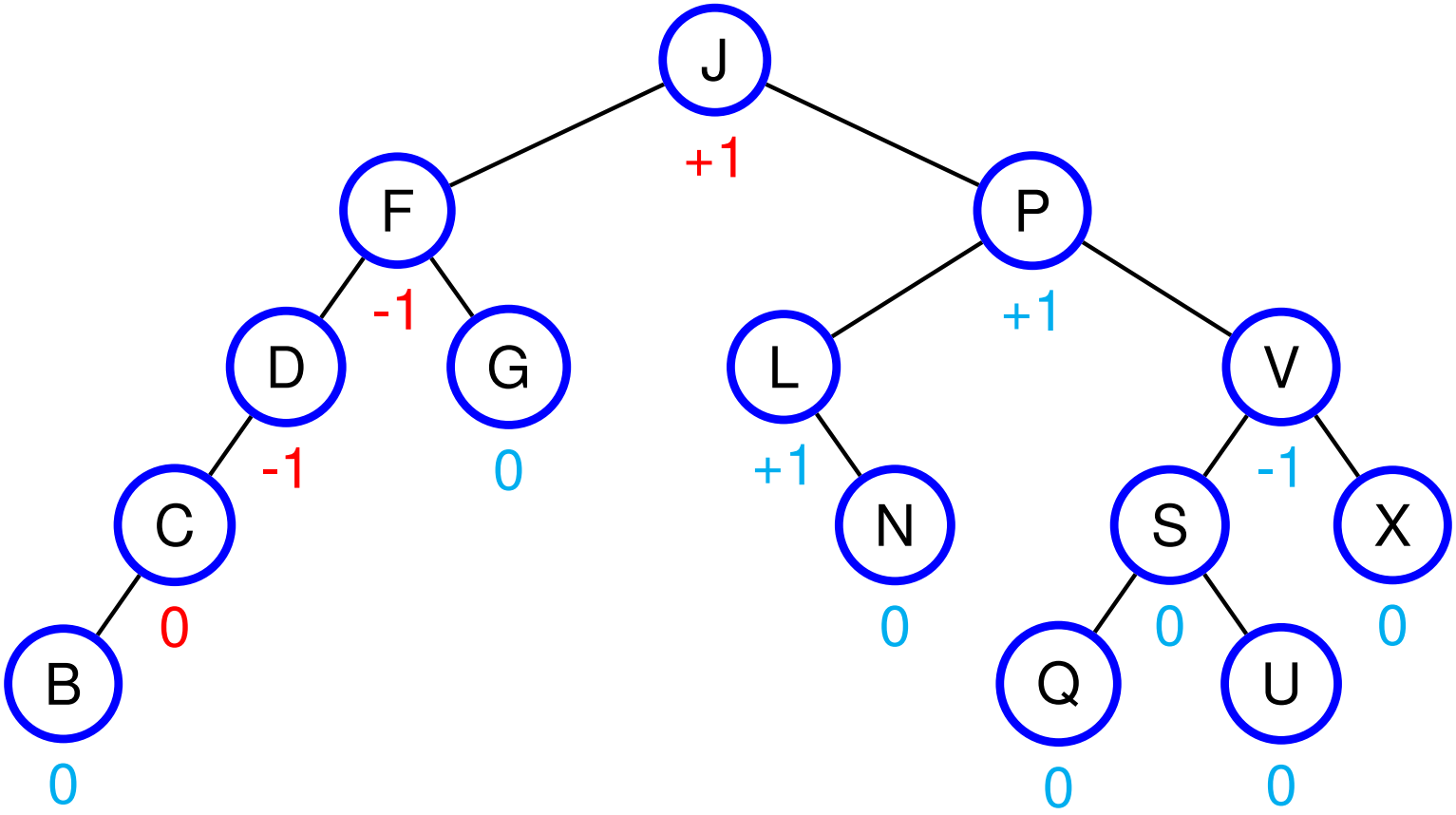
Another AVL insertion example

Let's insert B instead:



Another AVL insertion example

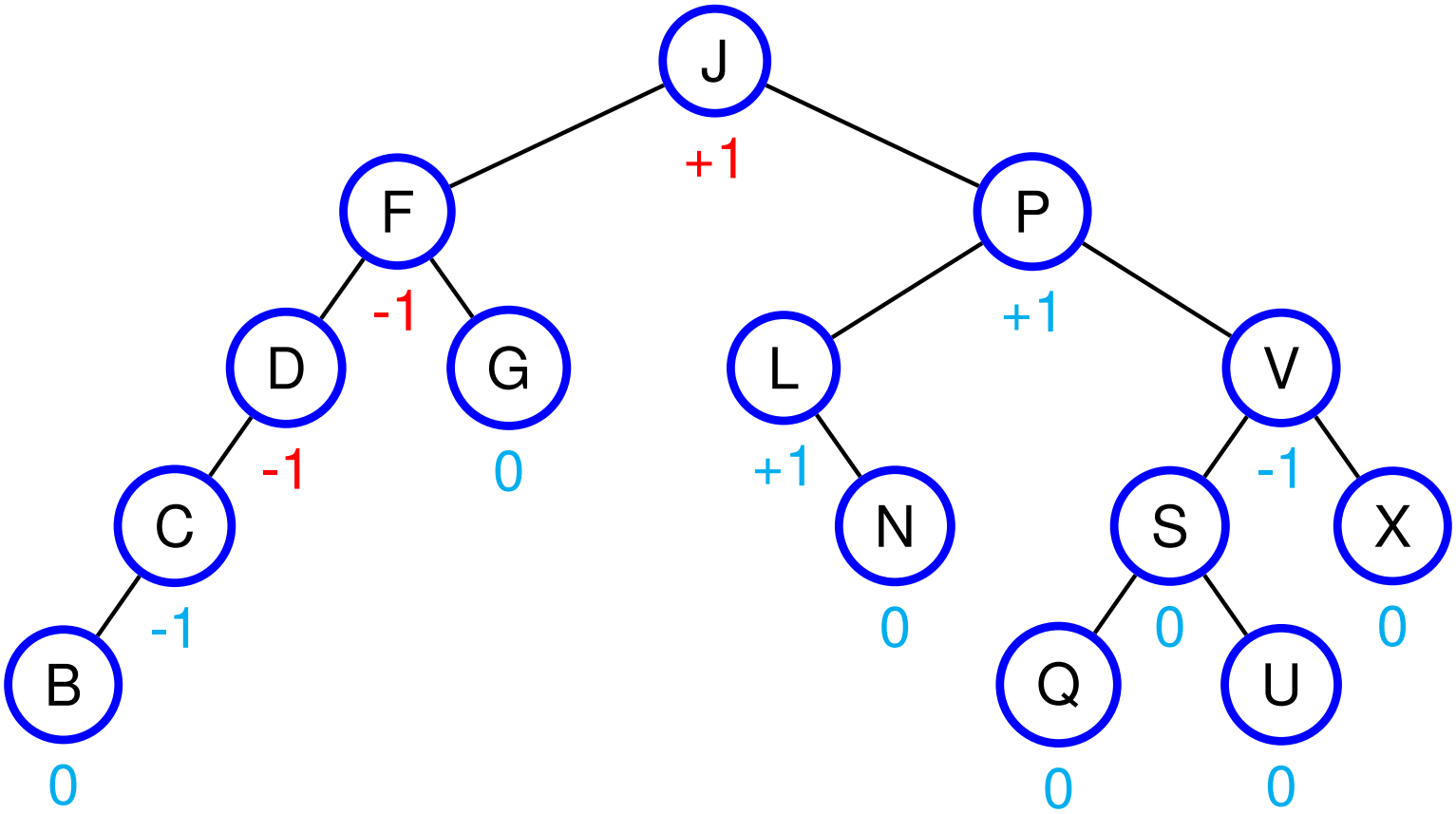
Let's insert B instead:



Leaf insert, invalidating balance factors along path.

Another AVL insertion example

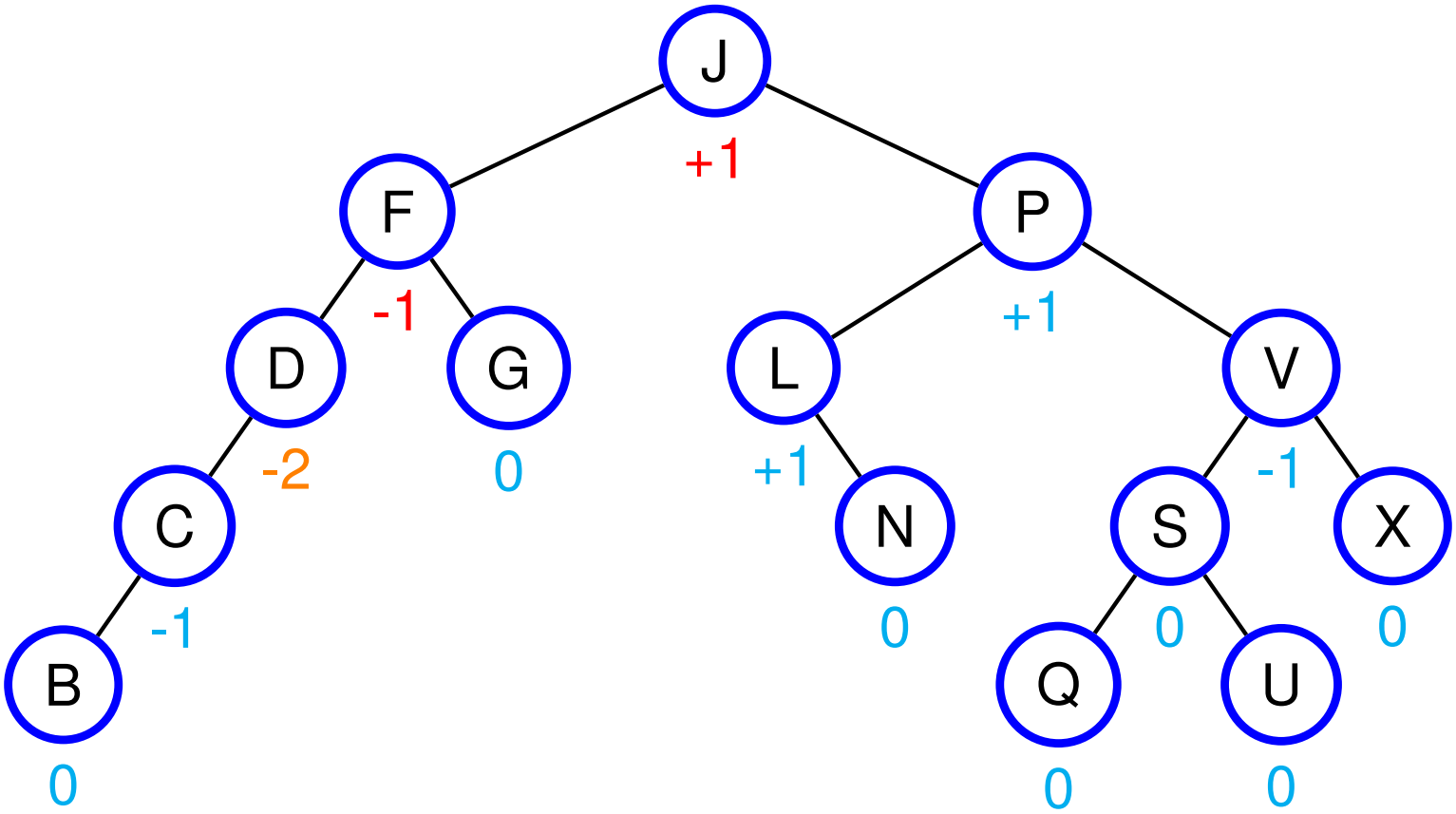
Let's insert B instead:



Left subtree got taller.

Another AVL insertion example

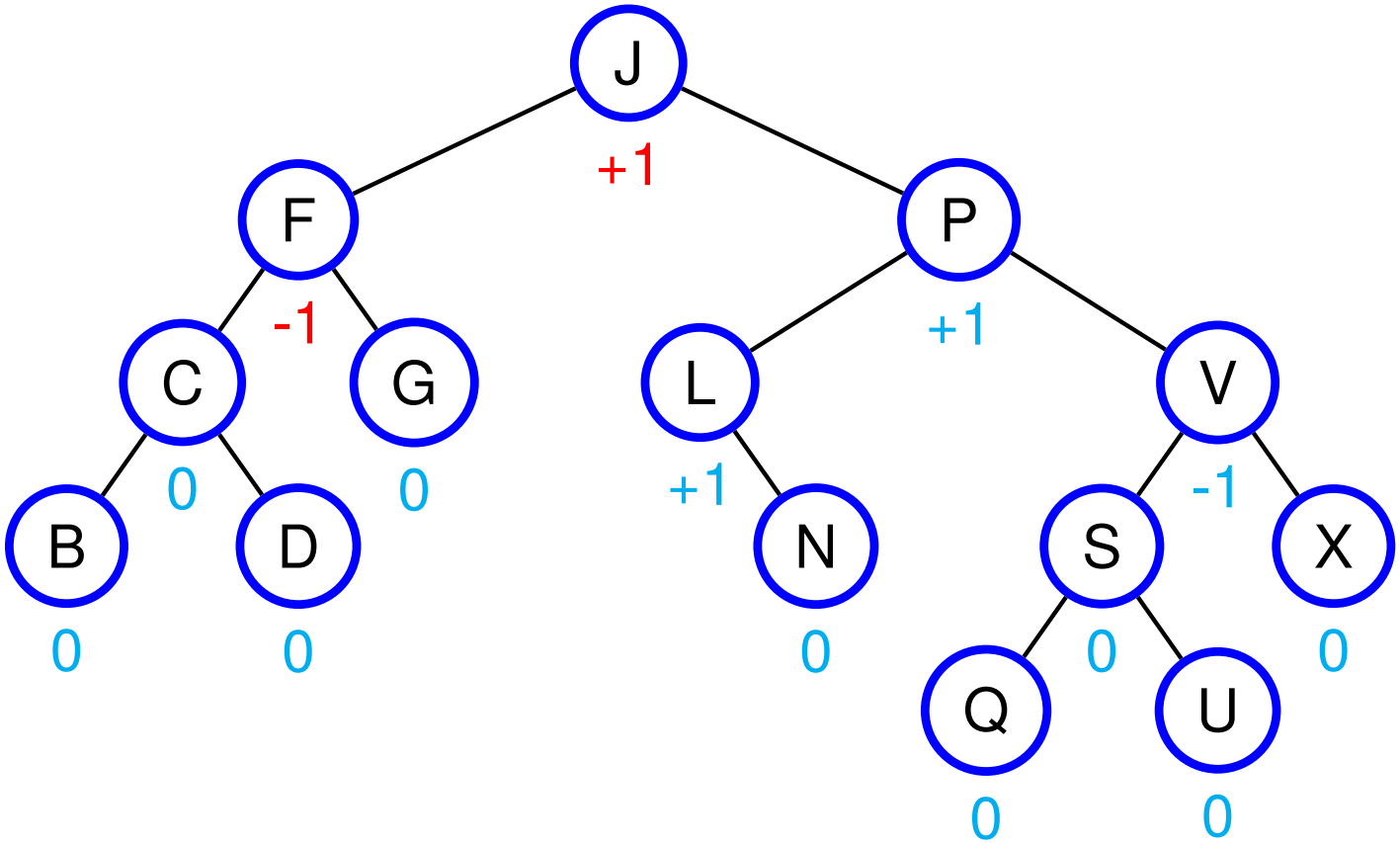
Let's insert B instead:



Left subtree at C also got taller. Outside $[-1, +1]$! Oops.

Another AVL insertion example

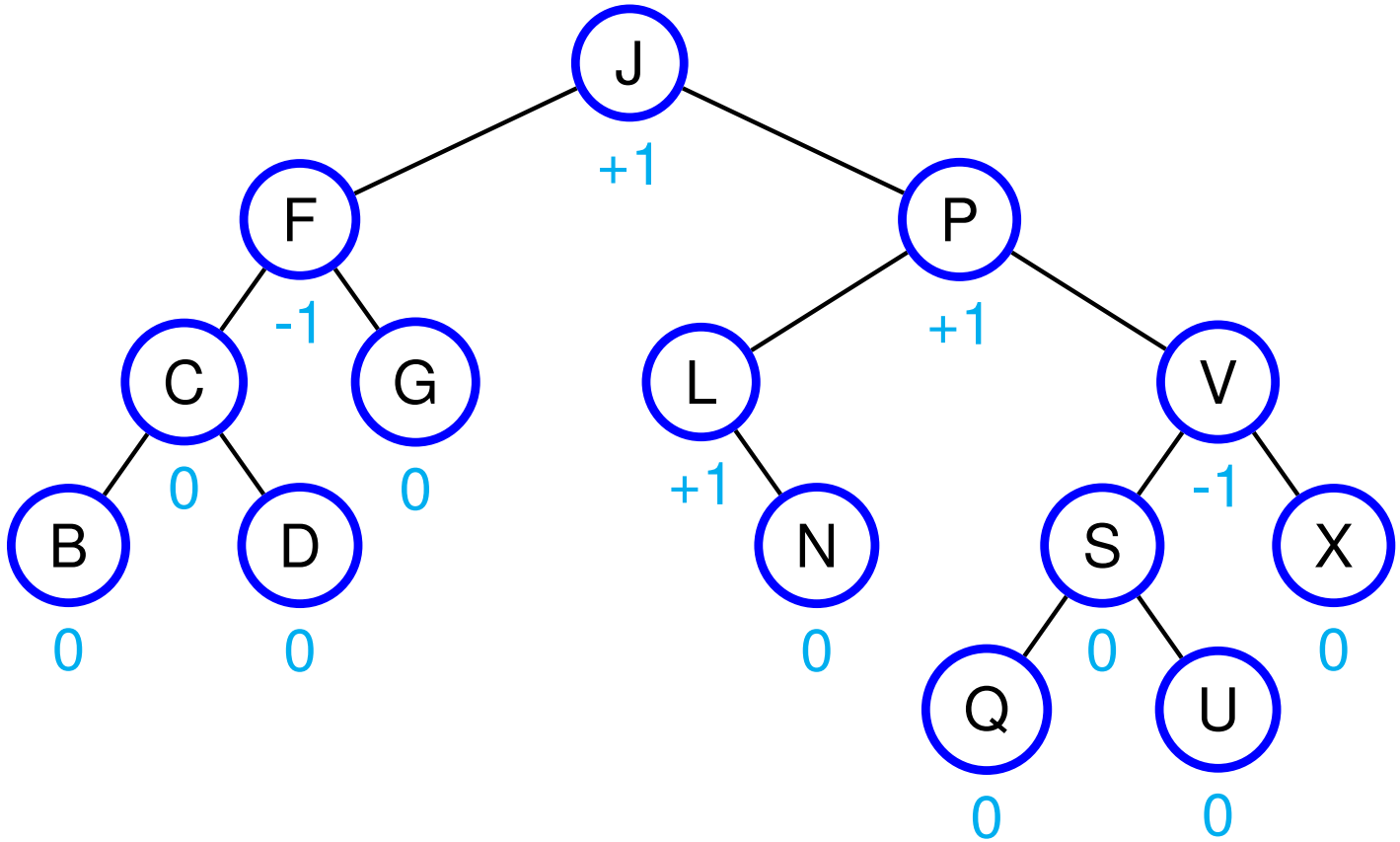
Let's insert B instead:



Rotate right centered at D to restore balance!

Another AVL insertion example

Let's insert B instead:

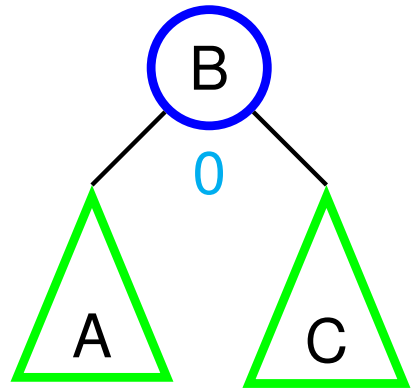


Left subtree of F was height 2 at start. Still height 2 now.

F subtree did not get taller. We're done.

Maintaining the AVL invariant

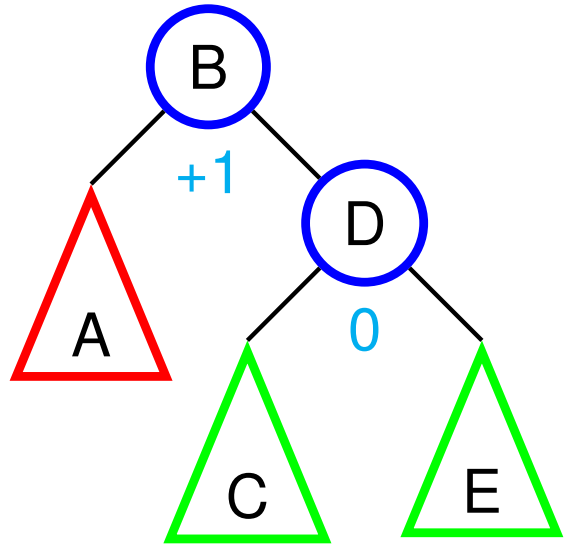
How can we do that in general? Let's look at all the cases.



Right now the balance factor is 0. Suppose we insert into A or C

- If that subtree grows taller, B's balance becomes -1 or +1.
- If that subtree doesn't grow taller, balance stays 0.
- In both cases, we're fine.

Maintaining the AVL invariant

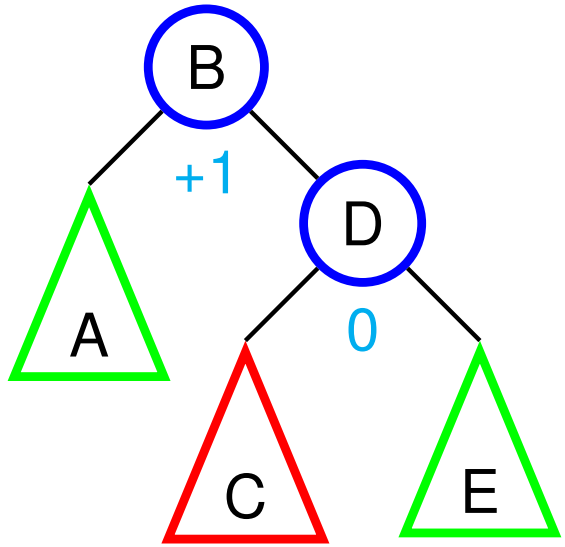


Right now the balance factor at B is +1.

Suppose we insert into A. What happens to B's balance factor?

- If no change in A's height then no change in B's balance.
- If A's height grows then B's balance factor goes to 0.
  I.e., insertion resulted in a more balanced tree!
- In both cases, we're fine.

Maintaining the AVL invariant

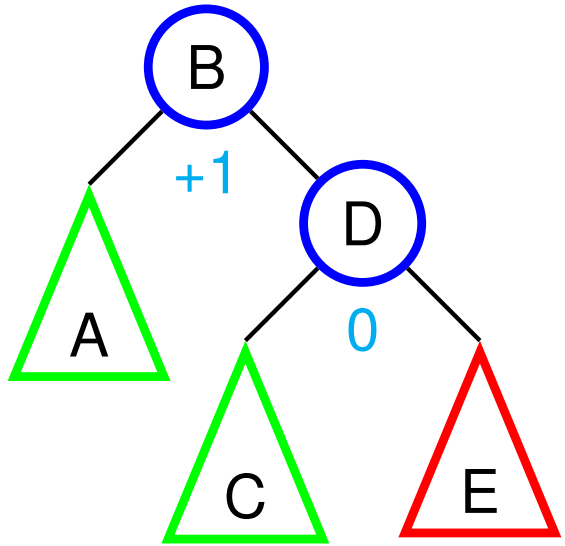


Right now the balance factor at B is +1.

Suppose we insert into C. What happens to B's balance factor?

- If no height change then B's balance doesn't change.
- If C grows then B's balance factor becomes +2.
  I.e., insertion pushed imbalance over the edge!
- Not ok!

Maintaining the AVL invariant



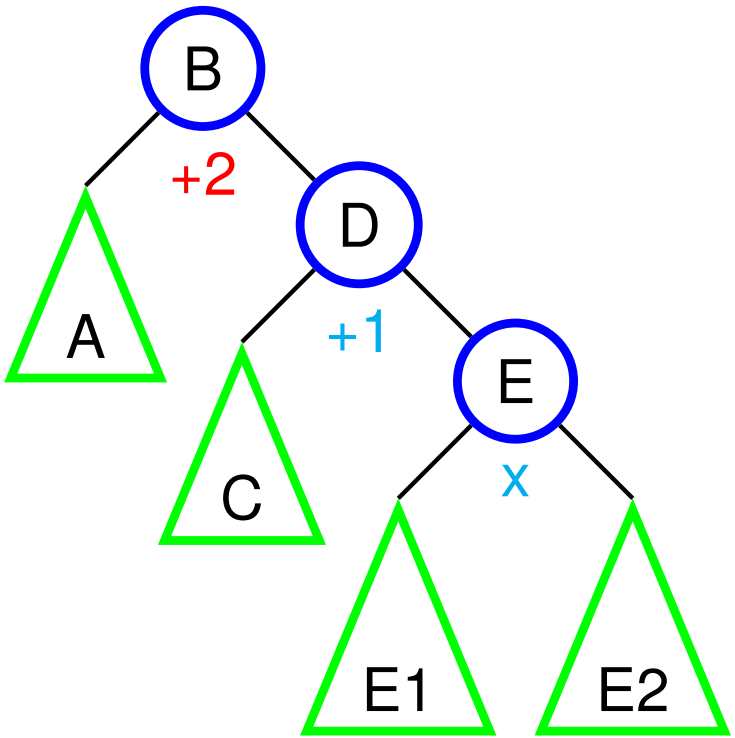
Right now the balance factor at B is +1.

Suppose we insert into E. What happens to B's balance factor?

- If no height change then B's balance doesn't change
- If E grows then B's balance factor becomes +2.
  I.e., insertion pushed imbalance over the edge!
- Not ok!

The right-right case

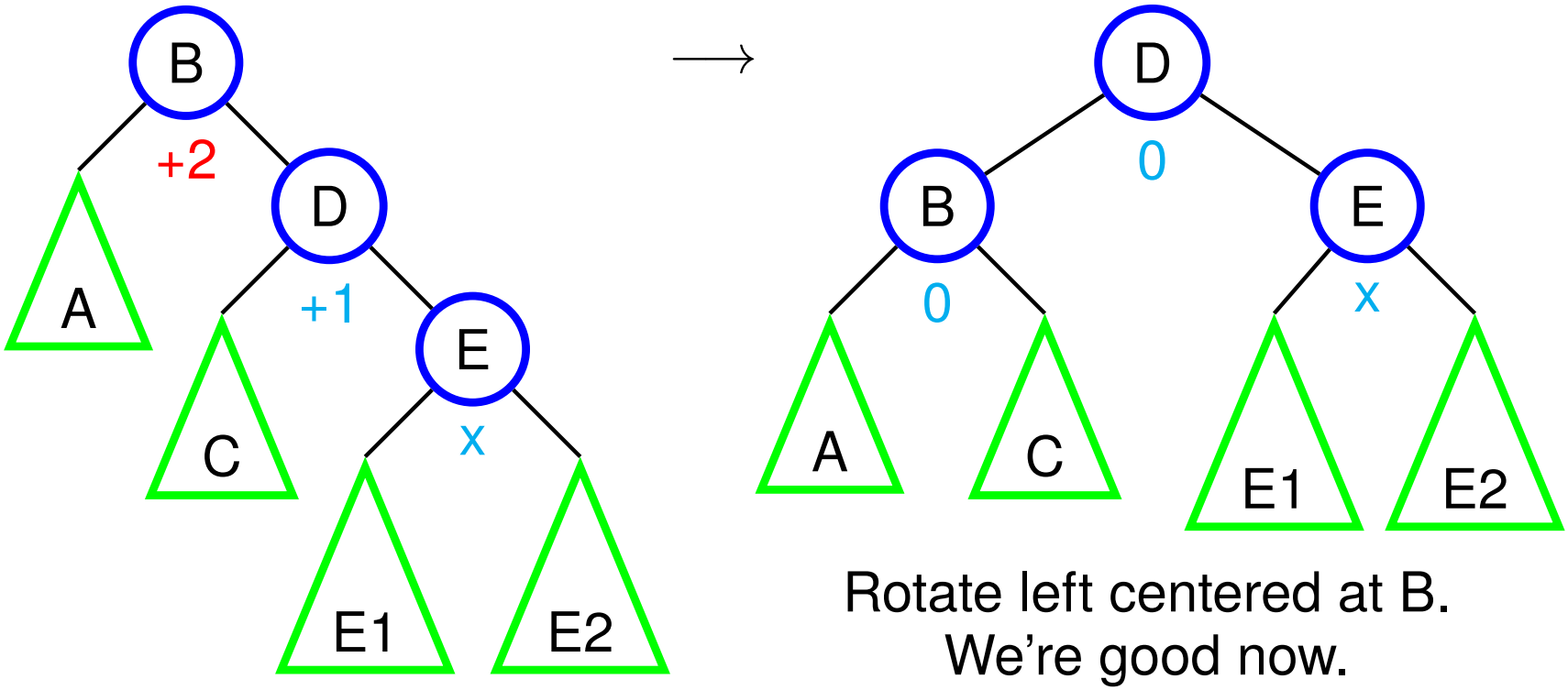
If the height of the right-right subtree (E) increases, we get this:



x = balance factor could be anything in $[-1, +1]$

The right-right case

If the height of the right-right subtree (E) increases, we get this:

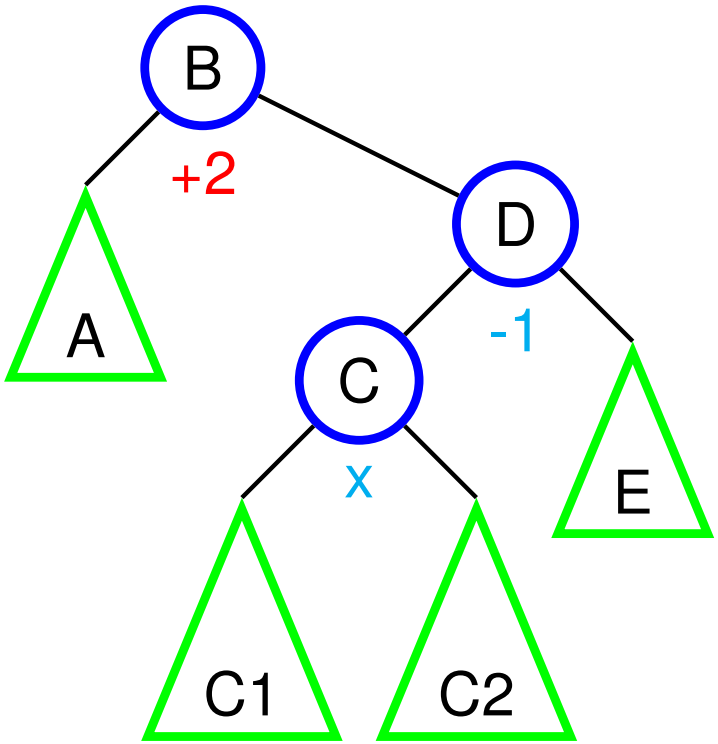


Rotate left centered at B.
We're good now.

x = balance factor could be anything in $[-1, +1]$

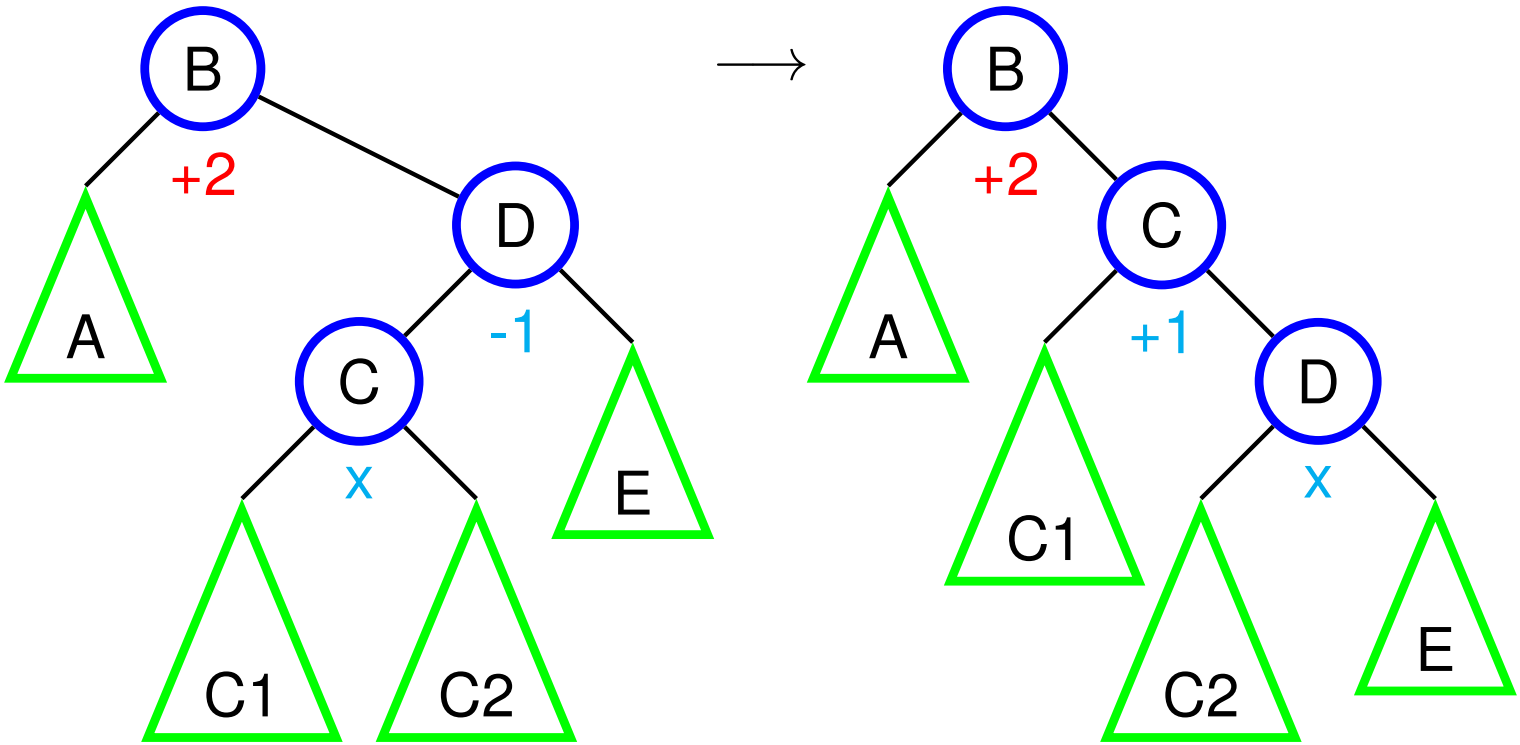
The right-left case

If the height of the right-left subtree (C) increases, we get this:



The right-left case

If the height of the right-left subtree (C) increases, we get this:



Rotate right centered at D.

This is now the right-right case, which we know how to handle!

Maintaining the AVL invariant

- We've seen the right-right and right-left cases
- The left-left and left-right cases are symmetrical

- Deletion is like ordinary BST deletion, but with the same repair cases as insertion

Self-Balancing BSTs beyond AVL

We wanted dictionaries with $\mathcal{O}(\log n)$ operations.
Mission accomplished, right?

What more could we possibly want?

Self-Balancing BSTs beyond AVL

We wanted dictionaries with $\mathcal{O}(\log n)$ operations.
Mission accomplished, right?

What more could we possibly want?

- AVL trees are very rigidly balanced; they don't tolerate much unbalance before needing repairs
  - Some other solutions (e.g., *Red-Black trees*) are looser
  - Tolerating more unbalance = gives time to sort itself out
  - Fewer rotations = better insertion/deletion constant factors

Self-Balancing BSTs beyond AVL

We wanted dictionaries with $\mathcal{O}(\log n)$ operations.
Mission accomplished, right?

What more could we possibly want?

- AVL trees are very rigidly balanced; they don't tolerate much unbalance before needing repairs
  - Some other solutions (e.g., *Red-Black trees*) are looser
  - Tolerating more unbalance = gives time to sort itself out
  - Fewer rotations = better insertion/deletion constant factors
- AVL trees (like any linked structures) have a lot of arrows
  - Not great for locality (Cf. Comp Sci 213) $\rightarrow$ performance
  - Some other solutions (e.g., *B trees*) use fewer arrows
  - Fewer arrows = better locality = better performance

Self-Balancing BSTs beyond AVL

We wanted dictionaries with $\mathcal{O}(\log n)$ operations.
Mission accomplished, right?

What more could we possibly want?

- AVL trees are very rigidly balanced; they don't tolerate much unbalance before needing repairs
  - Some other solutions (e.g., *Red-Black trees*) are looser
  - Tolerating more unbalance = gives time to sort itself out
  - Fewer rotations = better insertion/deletion constant factors

- AVL trees (like any linked structures) have a lot of arrows
  - Not great for locality (Cf. Comp Sci 213) $\rightarrow$ performance
  - Some other solutions (e.g., *B trees*) use fewer arrows
  - Fewer arrows = better locality = better performance

Lots of more advanced, more specialized solutions!

All of those have the same complexity: $\mathcal{O}(\log n)$
But better in different contexts!

Another cool thing about BSTs

- Wait, don't you need keys to be comparable (i.e., $>$)?
  - So how can we build general-purpose BST dictionaries?
  - Turns out we can turn any kind of key into a number. How?

Another cool thing about BSTs

- Wait, don't you need keys to be comparable (i.e., $>$)?
  - So how can we build general-purpose BST dictionaries?
  - Turns out we can turn any kind of key into a number. How?
  - Hash functions to the rescue!

- But if your keys *are* comparable, can do *range queries*!
  - Instead of: give me the value with key k
  - Give me a value whose key is less than k!
    - Or greater than, between, etc.
  - Useful for, e.g., databases (next week), finding large enough (but not too large) blocks of memory (malloc), etc.

If we have time: AVL trees in DSSL2

See BSTs in action

- See `avl.rkt`.
- `https://www.cs.usfca.edu/~galles/visualization/BST.html`
- `https://www.cs.usfca.edu/~galles/visualization/AVLtree.html`
  - He uses subtree height instead of balance factor, but isomorphic.
- He has other kinds of self-balancing trees too.

Next time: The relational model